Pontificia Universidad Javeriana
Facultad de Ingeniería — Ingeniería de Sistemas

Software Architecture Document
SAD

GammaLab v2.0

Proyecto	Plataforma de análisis y visualización de señales neurofisiológicas Gamma Lab
Documento	Software Architecture Document — SAD
Versión	1.0
Estado	En elaboración
Fecha	27 de agosto de 2026

Autores

Juan Felipe Romero Ortiz — `juanromeroo@javeriana.edu.co`
Laura Juliana Garzón Arias — `garzonlj@javeriana.edu.co`
Xiomara Herrera Suárez — `x_herrera@javeriana.edu.co`
Sebastián Almanza Galvis — `s.almanzag@javeriana.edu.co`

Nota de uso. Documento académico/interno del equipo de proyecto. No distribuir sin autorización.
Trazabilidad. Este SAD define la arquitectura del sistema y traza su relación con el SRS de GammaLab v2.0.

Índice

1. Introducción **7**

1.1. Propósito . . . 7

1.2. Alcance . . . 7

1.3. Definiciones y acrónimos . . . 7

1.4. Audiencia . . . 8

1.5. Referencias relacionadas . . . 8

2. Contexto del sistema **8**

2.1. Visión General . . . 8

2.1.1. Núcleo con pocas responsabilidades y contratos débiles . . . 9

2.1.2. Estado y memoria . . . 9

2.1.3. Concurrencia y procesos pesados . . . 10

2.1.4. Tolerancia a fallos y Observabilidad . . . 10

2.2. Limitaciones del sistema . . . 11

2.2.1. Software . . . 11

2.2.2. Hardware . . . 11

2.2.3. Datos . . . 11

3. Requisitos y atributos de calidad **12**

3.1. Requisitos de los Stakeholders . . . 12

3.1.1. Identificación de los interesados . . . 12

3.1.2. Cómo se interpreta el cuadro . . . 14

3.1.3. Criterios de éxito del PMP y su implicación arquitectónica . . . 14

3.2. Atributos de Calidad . . . 15

3.2.1. Marco de clasificación . . . 15

3.2.2. Rendimiento . . . 16

3.2.3. Confiabilidad . . . 17

3.2.4. Usabilidad . . . 17

3.2.5. Soporte . . . 18

3.2.6. Priorización y compromisos entre atributos . . . 18

3.3. Supuestos . . . 19

4. Principios y decisiones arquitectónicas **21**

4.1. Estilo arquitectónico: microkernel con núcleo activo . . . 22

4.2. Principios arquitectónicos . . . 24

4.2.1. Núcleo y contratos . . . 24
4.2.2. Estado y memoria . . . 24
4.2.3. Concurrencia . . . 25
4.2.4. Tolerancia a fallos y observabilidad . . . 25
4.3. Decisiones arquitectónicas . . . 25
4.3.1. Modelo de concurrencia del TaskOrchestrator . . . 27
4.4. Patrones aplicados . . . 28
4.5. Trazabilidad entre el diagnóstico y las decisiones . . . 29

5. Vistas Arquitectónicas (4+1) 29
5.1. Vista de Escenarios . . . 29
5.1.1. Propósito . . . 29
5.1.2. Notación . . . 29
5.1.3. Visión general del modelo . . . 30
5.1.4. Subsistema Interfaz y Usabilidad . . . 31
5.1.5. Subsistema Núcleo y Ciclo de Vida . . . 32
5.1.6. Subsistema Preprocesamiento . . . 34
5.1.7. Subsistema Análisis . . . 35
5.1.8. Subsistema Exportación . . . 36
5.2. Vista Lógica . . . 36
5.2.1. Propósito . . . 36
5.2.2. Notación . . . 37
5.2.3. Visión general del modelo . . . 37
5.2.4. Elementos del núcleo . . . 37
5.2.5. Servicios del sistema . . . 38
5.2.6. Elementos de presentación . . . 39
5.2.7. Contrato de extensión . . . 40
5.2.8. Extensiones . . . 40
5.2.9. Modelo de datos . . . 41
5.2.10. Relaciones estructurales . . . 41
5.2.11. Justificación de las decisiones . . . 42
5.2.12. Trazabilidad con los requerimientos . . . 43
5.3. Vista de despliegue . . . 43
5.3.1. Propósito . . . 43
5.3.2. Notación . . . 44
5.3.3. Visión general del modelo . . . 44

5.3.4. Componentes principales . . . 45
5.3.5. Servicios de soporte y gestión de datos . . . 46
5.3.6. Orquestador del núcleo . . . 47
5.3.7. Plugins funcionales . . . 47
5.3.8. Documentación de ayuda . . . 48
5.3.9. Flujo general . . . 48
5.4. Vista de Procesos . . . 49
5.4.1. Propósito . . . 49
5.4.2. Notación . . . 50
5.4.3. Visión general del modelo . . . 50
5.4.4. Inicio de la aplicación . . . 52
5.4.5. Apertura de proyecto o carga de una señal . . . 52
5.4.6. Procesamiento de señales . . . 52
5.4.7. Ejecución de análisis y coordinación de tareas . . . 52
5.4.8. Visualización e interacción con los resultados . . . 52
5.4.9. Continuación del trabajo, guardado y cierre . . . 52
5.5. Vista física . . . 52
5.5.1. Propósito . . . 52
5.5.2. Notación . . . 52
5.5.3. Visión general del modelo . . . 52
5.5.4. Componentes internos . . . 53
5.5.5. Artefactos externos . . . 54

6. Atributos de calidad 55

6.1. Escenarios de calidad . . . 55
6.1.1. Escenario de Calidad 1: Integración de un Plugin Externo . . . 55
6.1.2. Escenario de Calidad 2: Carga de Señal de Gran Tamaño . . . 56
6.1.3. Escenario de Calidad 3: Manejo de Errores en Tiempo de Ejecución . . . 56
6.1.4. Escenario de Calidad 4: Exportación de Datos o Gráficas . . . 57
6.1.5. Escenario de Calidad 5: Uso del Sistema sin Experiencia Técnica . . . 57
6.1.6. Escenario de Calidad 6: Rendimiento con Múltiples Señales Cargadas . . . 58
6.1.7. Escenario de Calidad 7: Resiliencia ante Archivos Corruptos o Incompatibles 58
6.1.8. Escenario de Calidad 8: Consistencia entre Datos Ingresados y Gráficas Generadas . . . 59
6.2. Reglas de compatibilidad y versionado . . . 59
6.3. Métricas y plan de verificación . . . 60
6.4. Plan de evidencia . . . 61

6.5. Conclusión 61

7. Trazabilidad de requerimientos y casos de uso **62**

7.1. Fuentes y criterio de construcción 62

7.2. De los casos de uso a los requerimientos 62

7.3. De los requerimientos a los casos de uso 63

7.4. De los casos de uso a las decisiones arquitectónicas 65

7.5. Análisis de cobertura 66

8. Anexos **66**

8.1. Anexo A — Relación con el SRS 66

8.2. Anexo B — Relación con el SDD 66

8.3. Anexo C — Trazabilidad global de artefactos 67

8.4. Anexo D — Diagramas y tablas de referencia 67

9. Referencias **67**

Índice de figuras

1. Arquitectura microkernel de Gamma Lab 2.0: el núcleo y sus servicios, la capa de presentación y las extensiones conectadas mediante el contrato `IPlugin`. 23

2. Diagrama de casos de uso del sistema completo Gamma Lab 2.0 (23 CU en 5 subsistemas). Las relaciones *include*/*extend* se tomaron literalmente del campo "Extensiones" de cada ficha; ninguna es inferida por similitud temática. 31

3. Diagrama de clases del sistema Gamma Lab 2.0. Los elementos en verde se encuentran implementados y los elementos en rojo están planificados. 37

4. Vista de desarrollo de GammaLab 2.0: componentes organizados por paquetes funcionales (`io`, `preprocessing`, `analysis`, `measure` y `faq`), junto con los componentes transversales de la arquitectura. 45

5. Flujo maestro de sesión de Gamma Lab 2.0, con carriles Investigador/Sistema. Verde = ya existe, rojo = nuevo, blanco = mixto. 51

6. Vista física de Gamma Lab 2.0: estación única de escritorio y su superficie de E/S. 53

Índice de cuadros

1. Acrónimos y términos utilizados en el documento . . . 7
2. Stakeholders de GammaLab v2.0, rol según el PMP y necesidades que condicionan la arquitectura . . . 13
3. Matriz de interés de los stakeholders por atributo de calidad . . . 14
4. Criterios de éxito del PMP con impacto arquitectónico . . . 14
5. Resumen de los atributos de calidad de GammaLab v2.0 . . . 15
6. Compromisos entre atributos de calidad y decisión adoptada . . . 19
7. Supuestos de GammaLab v2.0, su fuente y su impacto . . . 19
8. Síntesis del diagnóstico de Gamma Lab 1.0 . . . 22
9. Decisiones arquitectónicas de Gamma Lab 2.0 . . . 27
10. Patrones arquitectónicos aplicados en Gamma Lab 2.0. . . . 28
11. Trazabilidad entre el diagnóstico, los principios y las decisiones . . . 29
12. Casos de uso del subsistema Interfaz y Usabilidad . . . 32
13. Casos de uso del subsistema Núcleo y Ciclo de Vida . . . 33
14. Casos de uso del subsistema Preprocesamiento . . . 34
15. Casos de uso del subsistema Análisis . . . 35
16. Casos de uso del subsistema Exportación . . . 36
17. Extensiones representadas en la vista lógica. . . . 40
18. Relaciones estructurales de la vista lógica. . . . 42
19. Trazabilidad entre elementos de la vista lógica y requerimientos. . . . 43
20. Escenario de calidad 1 — Integración de un plugin externo . . . 55
21. Escenario de calidad 2 — Carga de señal de gran tamaño . . . 56
22. Escenario de calidad 3 — Manejo de errores en tiempo de ejecución . . . 56
23. Escenario de calidad 4 — Exportación de datos o gráficas . . . 57
24. Escenario de calidad 5 — Operación por un usuario sin experiencia técnica . . . 57
25. Escenario de calidad 6 — Rendimiento con múltiples señales cargadas . . . 58
26. Escenario de calidad 7 — Resiliencia ante archivos corruptos o incompatibles . . . 58
27. Escenario de calidad 8 — Consistencia entre datos ingresados y gráficas generadas 59
28. Versiones de dependencias externas ancladas en `requirements.txt` . . . 60
29. Métricas, método de verificación y criterio de aceptación por atributo de calidad . 60
30. Requerimientos realizados por cada caso de uso . . . 63
31. Casos de uso que realizan cada requerimiento . . . 65
32. Decisiones arquitectónicas que sostienen cada caso de uso . . . 65
33. Cobertura de la trazabilidad entre requerimientos y casos de uso . . . 66

1. Introducción

1.1. Propósito

El presente documento tiene como propósito definir y documentar la arquitectura de GammaLab v2.0, estableciendo los principios, decisiones y lineamientos que orientan su evolución respecto a GammaLab v1.0. Para ello, describe la estructura arquitectónica de la solución, sus componentes principales, las relaciones entre ellos y las decisiones que permiten soportar la incorporación de nuevas capacidades de análisis de señales EEG, mejorar el rendimiento del sistema y facilitar su evolución y extensibilidad. Asimismo, presenta las vistas arquitectónicas bajo el modelo 4+1 y establece los criterios necesarios para evaluar que la arquitectura propuesta responda a los atributos de calidad definidos para el sistema, manteniendo coherencia con los requerimientos y objetivos establecidos para GammaLab v2.0.

1.2. Alcance

Este documento abarca la definición de los principios y decisiones arquitectónicas que rigen la estructura de Gamma Lab v2.0. La arquitectura adopta un estilo Microkernel, compuesto por un núcleo de servicios esenciales y un conjunto de plugins que extienden las capacidades del sistema mediante componentes desacoplados. Asimismo, se describen las vistas arquitectónicas bajo el modelo 4+1, los componentes y sus relaciones, los atributos de calidad, las estrategias para alcanzarlos y las métricas utilizadas para su evaluación. Finalmente, se documentan los principales riesgos y decisiones arquitectónicas y su trazabilidad con el SRS de Gamma Lab.

Fuera del alcance de este documento quedan los aspectos relacionados con la administración del proyecto, la implementación detallada de los componentes, los procedimientos de pruebas y las instrucciones de uso del sistema, los cuales se encuentran documentados en los artefactos correspondientes.

1.3. Definiciones y acrónimos

Cuadro 1: Acrónimos y términos utilizados en el documento

Sigla / Término	Definición
SAD	Software Architecture Document. Documento de arquitectura del sistema.
SRS	Software Requirements Specification. Documento de especificación de requisitos.
SDD	Software Design Document. Documento de diseño técnico.
IoC	Inversión de Control. Principio que delega el flujo de ejecución a un contenedor.
DI	Dependency Injection. Técnica para resolver dependencias desde un contenedor externo.
MVP / MVC	Patrones de separación de interfaz y lógica (Model–View–Presenter / Model–View–Controller).
Template Method	Patrón que define un flujo general con pasos sobrescribibles.
Adapter	Patrón para estandarizar interfaces incompatibles.
Caching	Estrategia para evitar recalcular o recomponer estructuras de datos.

Sigla / Término	Definición
Python	Lenguaje de programación principal de *Gamma Lab*.
PyQt5	Framework de interfaz gráfica basado en Qt para Python.
VTK	Visualization Toolkit. Biblioteca para visualización científica 2D/3D.
QVTK	Interfaz `QVTKRenderWindowInteractor` para incrustar VTK en aplicaciones Qt.
NumPy	Librería para cálculo numérico vectorizado.
EDF	European Data Format. Formato estándar para señales biomédicas.
ABF	Axon Binary File. Formato de archivo utilizado por Axon Instruments para registros biomédicos.
GUI Toolkit	Conjunto de herramientas para construir interfaces gráficas (Qt, PyQt, VTK).

1.4. Audiencia

Este documento está dirigido al equipo de desarrollo, así como a cualquier stakeholder con formación técnica que requiera comprender las decisiones arquitectónicas del sistema. Asimismo, está orientado a todos los investigadores y personas del Laboratorio de Electrofisiología de la Universidad de Colombia que estén interesados en conocer la estructura interna, los principios de diseño y los mecanismos de GammaLab v2.0 para el uso, mantenimiento y futuros cambios.

1.5. Referencias relacionadas

Este SAD se alinea con:

- **SRS de Gamma Lab**: requisitos funcionales y no funcionales del sistema.

Las referencias técnicas externas (PyQt, VTK, NumPy, EDF/ABF, YAML).

2. Contexto del sistema

2.1. Visión General

Gamma Lab está construido sobre una arquitectura de microkernel, en la que un núcleo mínimo (Kernel) coordina la aplicación y la funcionalidad de dominio se delega en plugins independientes. El núcleo actúa como registro de servicios y de extensiones, y ofrece además un canal de eventos para comunicar los componentes. Los 16 plugins actuales se descubren automáticamente durante el arranque a partir de un descriptor declarativo (`properties.yml`). Cada uno implementa el contrato común `IPlugin`, y todos acceden al estado de la aplicación a través de un servicio compartido, el `DataStore`.

Teniendo en cuenta que Gamma Lab 2.0 parte de una base funcional y arquitectónica ya construida, se llevó a cabo un diagnóstico de la arquitectura inicial con el propósito de evaluar su estado actual, identificar las limitaciones de la implementación existente y reconocer oportunidades de mejora. Los resultados de este análisis se presentan en las siguientes secciones.

2.1.1. Núcleo con pocas responsabilidades y contratos débiles

En Gamma Lab 1.0 el núcleo se limita a ser un registro pasivo: mantiene dos diccionarios, uno de plugins y otro de servicios, y un canal de eventos genérico. No asume responsabilidades propias del estilo microkernel, como gestionar el ciclo de vida de las extensiones, mediar su comunicación o verificar sus dependencias. El resultado es un *anemic kernel*: el núcleo existe estructuralmente, pero no gobierna el comportamiento del sistema en tiempo de ejecución.

La carencia más visible es la ausencia de gestión del ciclo de vida. Aunque `Kernel` dispone de `stop_plugins()`, ningún componente lo invoca y no existen estados ni transiciones que permitan controlar las extensiones. En consecuencia, el cierre de la aplicación no garantiza la liberación ordenada de recursos como contextos gráficos e hilos en ejecución, ni es posible descargar o recargar una extensión sin reiniciar el sistema.

La comunicación presenta una limitación similar. El núcleo ofrece un canal de eventos basado en señales de Qt, pero su firma genérica (`str, object`) permite tópicos arbitrarios y cargas útiles sin tipo definido. Además, solo transporta dos eventos, por lo que la coordinación restante se resuelve mediante referencias directas entre componentes. El mecanismo de eventos, que debería contribuir al desacoplamiento, termina teniendo un papel marginal.

En cuanto a los contratos, `IPlugin` concentra la lógica de procesamiento, la construcción de la interfaz gráfica y el acceso a los datos, violando el principio de responsabilidad única. Al quedar el cálculo ligado a objetos de presentación, no puede probarse ni ejecutarse de forma aislada, por ejemplo en un hilo de trabajo o proceso independiente.

Finalmente, los servicios se obtienen mediante un *service locator* basado en cadenas literales (`get_service("DataStore")`), por lo que las dependencias quedan ocultas en la implementación y no se verifican durante la carga. Un error tipográfico o un servicio ausente se manifiesta únicamente en tiempo de ejecución. De forma similar, registrar un plugin con un nombre existente sobrescribe silenciosamente al anterior, permitiendo que errores de configuración permanezcan ocultos hasta que se utiliza la funcionalidad afectada.

2.1.2. Estado y memoria

El estado de la aplicación reside en el `DataStore`, un diccionario global y mutable al que cualquier plugin puede acceder y modificar. Como ningún componente es responsable del ciclo de vida, cada archivo abierto permanece en memoria hasta el cierre de la aplicación, junto con sus datos derivados. En consecuencia, el consumo de memoria crece de forma monótona durante la sesión, y el usuario, tras abrir varios archivos o repetir segmentaciones, percibe un deterioro progresivo del rendimiento que solo se resuelve reiniciando el programa.

A esta retención se suma la replicación de los datos. Las señales se cargan completas en memoria, sin acceso parcial ni bajo demanda, y buena parte del flujo opera en doble precisión (`float64`). Sobre esa base se generan copias adicionales:

- La conversión a una tabla de VTK para la visualización realiza una copia profunda de la señal y la conserva asociada al conjunto de datos.
- La caché de ensayos filtrados crea una nueva matriz cada vez que se descartan ensayos, aunque el código asume lo contrario.
- La lectura de archivos EDF con frecuencias de muestreo heterogéneas remuestrea cada canal sobre un nuevo arreglo.

Esta mutabilidad sin control compromete la consistencia del estado. El proceso de remoción de artefactos modifica la matriz de ensayos en el mismo lugar (*in-place*), desde un hilo secundario y accediendo a un atributo privado de `SignalDataset`, lo que rompe su encapsulamiento. Como la caché de ensayos filtrados solo se invalida cuando cambia el conjunto de descartes, puede seguir entregando una copia anterior a la corrección. En ese caso, los análisis posteriores operarían sobre datos que ya no corresponden a los que el usuario corrigió, sin que nada lo advierta. Además, como la modificación es destructiva y no conserva versiones previas, el usuario no puede deshacerla.

Por último, los recursos gráficos siguen la misma lógica de retención. Cada plugin crea su propia ventana de renderizado VTK sobre un contexto OpenGL compartido, y la ventana principal conserva todos los widgets instanciados hasta el cierre de la aplicación. Al desactivarse, un plugin solo deshabilita la interacción de su vista y no libera los buffers gráficos ni las tablas de datos asociadas, por lo que cada análisis que el usuario abre aumenta de forma permanente el consumo de memoria de video y de sistema.

2.1.3. Concurrencia y procesos pesados

GammaLab 1.0 no define un modelo de concurrencia a nivel arquitectónico. Por defecto, todo el procesamiento se ejecuta de forma síncrona en el hilo principal, el mismo que atiende el bucle de eventos de la interfaz gráfica. Por lo que, ante operaciones de varios segundos o minutos, el usuario no puede distinguir un procesamiento en curso de un fallo.

La concurrencia aparece solo como una decisión local de cada plugin. De los 16 plugins, únicamente dos delegan su cálculo en un hilo secundario, y cada uno lo hace con un patrón distinto:

- El promedio de *wavelets* hereda de `QThread`.
- La remoción de artefactos traslada un objeto de trabajo con `moveToThread`.

No existe una abstracción común para ejecutar tareas, ni un orquestador que las encole o limite cuántas se ejecutan simultáneamente. El sistema no impide que el usuario lance en paralelo varios procesos costosos desde distintos plugins. Cuando eso ocurre, compiten por los mismos núcleos de CPU y sus picos de memoria se suman, lo que agrava las condiciones de agotamiento descritas en la sección anterior.

La cancelación está igualmente limitada. Solo el promedio de *wavelets* la contempla, de forma cooperativa y entre un ensayo y el siguiente. Su rutina de limpieza invoca `quit()`, que no tiene efecto sobre un hilo sin bucle de eventos propio, y luego `wait()` desde el hilo principal, lo que congela la interfaz hasta que termine el ensayo en curso. Además, esa rutina está deshabilitada en el método de detención del plugin. Si el usuario cierra el plugin o la aplicación durante el cálculo, el hilo sigue ejecutándose sobre objetos que pueden estar siendo destruidos, y el cierre puede terminar en un error en lugar de una finalización ordenada.

Por último, el sistema carece de control de admisión. Antes de ejecutar un proceso no se estima su costo en tiempo ni en memoria, lo que puede generar que una configuración que excede los recursos del equipo no se rechace ni se advierta. Se ejecuta hasta congelar la aplicación o hasta fallar por falta de memoria, a veces tras varios minutos de cálculo cuyo resultado se pierde por completo.

2.1.4. Tolerancia a fallos y Observabilidad

Gamma Lab se ejecuta en un único proceso sin fronteras de aislamiento entre ellos. Un error nativo en VTK, un `MemoryError` o una excepción no capturada dentro de un *slot* de Qt terminan con la

aplicación completa, porque no existe un manejador global (`sys.excepthook`) que lo impida. El usuario pierde así todas las señales, segmentaciones y correcciones de la sesión. El cargador omite en silencio los plugins que fallan al importarse, y el núcleo mantiene registrados aquellos cuya inicialización falló. Los errores no se corrigen, solo se ocultan: el sistema continúa en un estado inconsistente y el usuario se encuentra con funcionalidades ausentes o con comportamientos erráticos, sin una explicación.

Estas fallas son difíciles de diagnosticar por la ausencia de observabilidad. El sistema no utiliza un mecanismo de registro estructurado. La información se emite con llamadas a `print`, sin niveles de severidad, marcas de tiempo ni identificación del componente de origen. Además, como la aplicación se ejecuta desde una consola, esa salida no se persiste y se pierde justamente cuando ocurre un cierre inesperado. Ante un fallo reportado no existe un registro que permita reconstruir qué operación se ejecutaba ni en qué estado estaba el sistema, por lo que el diagnóstico depende de reproducir manualmente el error.

2.2. Limitaciones del sistema

2.2.1. Software

Las siguientes condiciones tecnológicas forman parte del punto de partida del sistema:

- GammaLab está desarrollado en Python y se ejecuta sobre CPython, su intérprete de referencia.
- La interfaz gráfica está construida con el *framework* Qt, mediante su enlace oficial para Python, PySide6.
- La visualización de señales y resultados se realiza con la librería VTK, que utiliza OpenGL como motor de renderizado.
- La arquitectura base del sistema es un microkernel, en el que un núcleo central coordina un conjunto de plugins.
- Los plugins existentes se rigen por un contrato común de extensión, la interfaz `IPlugin`.
- La aplicación se distribuye como un instalador independiente exclusivo para Windows de 64 bits. No contempla versiones web, móviles ni para sistemas Unix o macOS.
- La aplicación es de uso individual. No incorpora control de acceso multiusuario, autenticación en red ni mecanismos de colaboración simultánea.

2.2.2. Hardware

El sistema debe operar sobre la infraestructura con la que hoy cuenta el laboratorio:

- Los equipos disponen de una capacidad de memoria RAM limitada.
- Los equipos cuentan con un número limitado de núcleos de procesamiento.
- La aceleración gráfica depende de la GPU y de los controladores OpenGL instalados en cada equipo.
- La aplicación no se conecta con equipos de adquisición, como amplificadores, electrodos o diademas inalámbricas.

2.2.3. Datos

Las características de los datos que procesa el sistema también condicionan su diseño:

- El sistema trabaja exclusivamente con señales previamente registradas. No adquiere ni procesa señales EEG en tiempo real.

- Las señales neurofisiológicas se reciben en formatos binarios ABF, EDF, BDF y MAT.
- Cada señal se carga completa en memoria al momento de abrirse.
- Los datos numéricos se representan en doble precisión (`float64`).
- Los registros EDF pueden contener canales con frecuencias de muestreo distintas entre sí.
- El estado de la aplicación se conserva en un repositorio compartido en memoria, el `DataStore`.
- La persistencia de los proyectos es local e individual, en el equipo del usuario. No se utilizan bases de datos centralizadas, servicios en la nube ni servidores remotos.
- Los datos de las señales se almacenan sin mecanismos de cifrado.

3. Requisitos y atributos de calidad

Una arquitectura no se justifica por sí misma, sino por las necesidades que debe satisfacer. Este capítulo establece ese punto de partida: identifica a las personas y entidades con interés en GammaLab v2.0 y traduce sus expectativas en requisitos que condicionan la estructura del sistema (Sección 3.1), define las cualidades que la solución debe exhibir y cómo se miden (Sección 3.2) y declara las condiciones que se dan por ciertas durante el diseño (Sección 3.3).

Los stakeholders y sus roles provienen del Plan de Administración del Proyecto (PMP), mientras que los requisitos no funcionales provienen de la Especificación de Requisitos de Software (SRS). Así, cada decisión descrita en los capítulos siguientes puede rastrearse hasta una necesidad concreta de un interesado.

3.1. Requisitos de los Stakeholders

3.1.1. Identificación de los interesados

Un *stakeholder* es toda persona o entidad que afecta el desarrollo de GammaLab v2.0 o se ve afectada por él. La identificación que se presenta a continuación es coherente con la estructura organizacional y la distribución de roles definidas en el PMP: el Laboratorio de Electrofisiología de la Universidad Nacional de Colombia actúa como cliente, usuario final y validador; la directora del proyecto actúa como *Product Owner* y puente entre el cliente y el equipo; y el equipo de desarrollo, conformado por cuatro estudiantes con un integrante que asume funciones equivalentes a *Scrum Master*, ejecuta el trabajo técnico. A ellos se suman los revisores académicos del trabajo de grado, que evalúan el producto final, y la comunidad científica, beneficiaria indirecta por la naturaleza de código abierto del sistema (licencia MIT).

Cuadro 2: Stakeholders de GammaLab v2.0, rol según el PMP y necesidades que condicionan la arquitectura

Stakeholder	Rol en el proyecto	Necesidades e intereses	Atributos de calidad asociados
Laboratorio de Electrofisiología (Universidad Nacional de Colombia)	Cliente, usuario final y validador. Participa en las Fases 1, 2 y 4.	Una herramienta gratuita e intuitiva que no exija conocimientos de programación; abrir sus registros (`.abf`, `.edf`, `.mat`) y ejecutar análisis de frecuencia, tiempo-frecuencia y acoplamiento fase-amplitud (PAC) sin bloquear la interfaz; resultados precisos y confiables; poder guardar y retomar su trabajo; y documentación clara.	Usabilidad, Rendimiento, Confiabilidad.
Directora del proyecto	*Product Owner*. Representa al cliente y prioriza el *Product Backlog*.	Rigor científico de los algoritmos de análisis, cumplimiento de los estándares de calidad y alineación del producto con los objetivos del proyecto; retroalimentación oportuna sobre diseño, código y documentación, y supervisión técnica y académica.	Confiabilidad, Soporte.
Equipo de desarrollo (cuatro *Scrum Developers*, uno con funciones de *Scrum Master*)	Diseña, implementa, integra, prueba y documenta el sistema.	Una arquitectura extensible (microkernel y plugins) que permita incorporar nuevos análisis sin modificar el núcleo; comprender y reutilizar el código heredado de la versión 1.0; procesamiento en segundo plano; y código verificable mediante pruebas automatizadas.	Soporte, Rendimiento.
Revisores académicos del trabajo de grado	Evalúan el cumplimiento de los entregables y de los objetivos.	Verificar el cumplimiento de los requisitos y de la metodología, y poder constatar con evidencia cuantitativa la mejora de GammaLab v2.0 respecto a la versión 1.0.	Rendimiento, Confiabilidad (verificabilidad).
Comunidad científica y usuarios de software libre	Beneficiario indirecto (derivado de la licencia MIT definida en el PMP).	Reutilizar, adaptar y extender el código abierto; reproducibilidad de los resultados y transparencia metodológica.	Soporte, Confiabilidad.

3.1.2. Cómo se interpreta el cuadro

La distinción entre el Laboratorio y la directora es deliberada. El Laboratorio es quien usa el sistema y valida los resultados; la directora, en su rol de *Product Owner*, canaliza y prioriza esas necesidades ante el equipo. Por ello, los requisitos del Laboratorio se tratan como **requisitos primarios**, mientras que los de los demás interesados cumplen una función de gobierno (directora y revisores) o de sostenibilidad (equipo y comunidad). El equipo de desarrollo es, además, quien convive con las consecuencias de la arquitectura a largo plazo, razón por la cual su interés en la extensibilidad y la mantenibilidad pesa tanto como el del usuario final en la usabilidad.

La Cuadro 3 resume qué atributos de calidad importan a cada interesado. Un círculo lleno (●) indica un interés principal, derivado directamente de sus necesidades; un círculo vacío (○) indica un interés secundario.

Cuadro 3: Matriz de interés de los stakeholders por atributo de calidad

Stakeholder	Usabilidad	Rendimiento	Confiabilidad	Soporte
Laboratorio de Electrofisiología	●	●	●	○
Directora del proyecto (*Product Owner*)	○	○	●	●
Equipo de desarrollo	○	●	○	●
Revisores académicos	○	●	●	○
Comunidad científica	○	—	○	●

3.1.3. Criterios de éxito del PMP y su implicación arquitectónica

El PMP define criterios de éxito cuantificables para el proyecto. No todos son de naturaleza arquitectónica (por ejemplo, la entrega en la fecha acordada o la completitud de la documentación son compromisos de gestión), pero cuatro de ellos imponen condiciones directas sobre el diseño del sistema y se recogen en la Cuadro 4.

Cuadro 4: Criterios de éxito del PMP con impacto arquitectónico

Criterio de éxito (PMP)	Atributo	Implicación para la arquitectura
Mejora de al menos 20–30 % en los tiempos de procesamiento respecto a GammaLab v1.0, derivada de la paralelización.	Rendimiento	Las operaciones de cómputo deben ejecutarse fuera del hilo de la interfaz, a través de un servicio de tareas central (NFR-P-14, NFR-S-04), de modo que el trabajo pesado no compita con la interacción.
El módulo de acoplamiento fase-amplitud (PAC) opera sin errores críticos y produce resultados reproducibles y validables.	Confiabilidad	El análisis PAC se ejecuta como tarea cancelable (NFR-P-16) y el procesamiento no altera la señal ni los trials de origen (NFR-R-05), lo que permite repetir un análisis y obtener el mismo resultado.

Criterio de éxito (PMP)	Atributo	Implicación para la arquitectura
Los problemas de la versión 1.0 priorizados con el cliente quedan corregidos y validados por él.	Confiabilidad, Soporte	La liberación de recursos al salir de un módulo (NFR-P-02), la terminación de hilos (NFR-R-01) y el acceso único al estado compartido (NFR-R-02) atacan la inestabilidad heredada, mientras que una vía pública para modificar los trials (NFR-S-02) evita que el problema reaparezca por acoplamiento.
El usuario final confirma en la Fase 4 que el sistema es intuitivo, que los resultados son precisos y que las funcionalidades responden a sus necesidades.	Usabilidad, Confiabilidad	Se requiere retroalimentación visual del progreso, mensajes diferenciados por severidad, ayudas contextuales y resultados mostrados en la misma pantalla en que se solicitaron (NFR-U-01 a NFR-U-09).
Cobertura de pruebas unitarias de al menos 80 %.	Soporte	La lógica de negocio debe poder verificarse sin la interfaz gráfica: servicios compartidos con una única instancia inyectada desde el núcleo (NFR-S-03) y plugins que no acceden a estructuras internas de otros módulos (NFR-S-02).

3.2. Atributos de Calidad

3.2.1. Marco de clasificación

Los atributos de calidad describen *cómo* debe comportarse el sistema, a diferencia de los requisitos funcionales, que describen *qué* debe hacer. Son relevantes para este documento porque, de todos los requisitos, son los que más condicionan la arquitectura: la manera de ejecutar un cálculo pesado, de aislar una extensión defectuosa o de representar una señal de gran tamaño no se resuelve con código local, sino con decisiones estructurales.

La clasificación sigue el modelo FURPS+ adoptado por el SRS, donde cada requisito no funcional se identifica con el código **NFR-X-YY** (X es la categoría y YY el consecutivo). En esta iteración se especificaron 36 requisitos no funcionales distribuidos en cuatro categorías, que constituyen los atributos de calidad del sistema: Usabilidad (9), Rendimiento (17), Confiabilidad (5) y Soporte (5). La Cuadro 5 los sintetiza.

Cuadro 5: Resumen de los atributos de calidad de GammaLab v2.0

Atributo	Qué garantiza	Medidas representativas	Requisitos
Rendimiento	Que el sistema responda con fluidez y use los recursos de forma controlada, incluso con señales grandes.	Carga de señal ≤ 2 s; Wavelet ≤ 10 s; interfaz con respuesta $\leq 0{,}3$ s; ≥ 24 FPS con 3 canales; cambio de módulo ≤ 500 ms; PAC de 20×20 bandas ≤ 30 s; arranque ≤ 8 s.	NFR-P-01 a NFR-P-17

Atributo	Qué garantiza	Medidas representativas	Requisitos
Confiabilidad	Que el sistema sea estable en sesiones prolongadas y que los datos del investigador no se corrompan.	Terminación garantizada de hilos; un único escritor por dato; señal de origen inmutable; representaciones gráficas invalidadas al cambiar los datos.	NFR-R-01 a NFR-R-05
Usabilidad	Que un investigador sin formación en programación pueda operar el sistema sin ayuda.	Retroalimentación de progreso en toda operación de más de 1 s; mensajes por severidad; ayuda emergente en cada control; pantalla de bienvenida e inicio.	NFR-U-01 a NFR-U-09
Soporte	Que el sistema se pueda instalar, extender y mantener sin degradar lo existente.	Plugin nuevo registrable sin modificar el núcleo; vía pública única para los trials; servicios compartidos con instancia única; ejecutable instalable.	NFR-S-01 a NFR-S-05

Cada atributo se desarrolla a continuación en cuatro partes: por qué importa, qué se exige, cómo lo resuelve la arquitectura y cómo se verifica. Los escenarios detallados (estímulo, respuesta y medida) se presentan en la Sección 6, y las métricas con su método de verificación en la Cuadro 29.

3.2.2. Rendimiento

Por qué importa. Las señales neurofisiológicas son densas y los análisis de frecuencia y tiempo-frecuencia (Wavelet, PAC) son computacionalmente costosos. Si el cálculo se ejecuta en el mismo hilo que la interfaz, la aplicación deja de responder y el investigador no puede distinguir un cálculo largo de una caída. El rendimiento es, además, el atributo con el que los revisores académicos medirán la mejora frente a la versión 1.0.

Qué se exige. Los requisitos fijan umbrales verificables sobre el hardware de referencia del laboratorio. Entre los más condicionantes se encuentran: cargar una señal en $\leq$ 2 s (NFR-P-03), completar el Wavelet en $\leq$ 10 s (NFR-P-04), sostener al menos 24 FPS al interactuar con tres canales (NFR-P-05), mantener la interfaz responsiva en $\leq$ 0,3 s mientras corre un análisis en segundo plano (NFR-P-10) y completar un comodulograma PAC de hasta 20×20 pares de bandas en $\leq$ 30 s con posibilidad de cancelarlo (NFR-P-16). El consumo de memoria también está acotado: la memoria de una señal debe liberarse al cerrar el proyecto o al cargar otra (NFR-P-01, NFR-P-08) y los recursos gráficos deben liberarse al salir de cada módulo (NFR-P-02).

Cómo lo resuelve la arquitectura.

- **Ejecución fuera del hilo de la interfaz.** Toda operación de cómputo de los plugins se despacha a través de un servicio de tareas registrado en el núcleo, que gestiona la cola, la cancelación y la notificación de progreso (NFR-P-14, NFR-S-04).
- **Submuestreo solo visual.** Para conservar la fluidez al graficar, se limita a 2 000 puntos por serie y 200 curvas simultáneas, y se reduce únicamente la representación, nunca los datos (NFR-P-15).
- **Conversión de datos en bloque.** Las representaciones gráficas se construyen con operaciones vectorizadas y no recorriendo los datos elemento por elemento (NFR-P-13).

- **Reutilización de la representación.** La tabla gráfica de una señal se conserva mientras los datos no cambien, evitando reconstruirla en cada dibujo (NFR-R-04).

Cómo se verifica. Escenarios de Calidad 2 (carga de señal de gran tamaño) y 6 (múltiples señales cargadas), y las métricas de la Cuadro 29.

3.2.3. Confiabilidad

Por qué importa. Un análisis científico solo es útil si sus resultados son correctos y reproducibles, y una sesión de trabajo larga no debe perderse por el fallo de un componente. Esta preocupación proviene tanto del Laboratorio, que debe poder confiar en los resultados, como de la directora, responsable de validar el rigor del producto. Además, el PMP identifica la estabilidad técnica como una de las limitaciones principales de la versión 1.0 que esta iteración debe corregir.

Qué se exige. El sistema debe garantizar la terminación y liberación de todo hilo trabajador cuando se cancela una tarea o se detiene el plugin que la originó (NFR-R-01); sincronizar el acceso al estado compartido de la sesión de modo que un solo propietario escriba cada dato (NFR-R-02); retirar los detectores de eventos de las gráficas al salir de cada módulo (NFR-R-03); descartar de forma explícita las representaciones gráficas cuando los datos cambian, para no mostrar una versión obsoleta (NFR-R-04); y no modificar jamás la señal cargada ni los trials originales (NFR-R-05).

Cómo lo resuelve la arquitectura.

- **Propiedad única de los datos.** El *DataStore* centraliza el estado de la sesión y se accede a él con un único escritor, de modo que la concurrencia no produzca estados inconsistentes (NFR-R-02).
- **Datos de origen inmutables.** Todo preprocesamiento produce un conjunto derivado y conserva disponible el original, de modo que cualquier análisis puede repetirse desde la fuente (NFR-R-05).
- **Ciclo de vida explícito de los plugins.** Cada plugin libera sus recursos gráficos, observadores y tareas al detenerse, en lugar de limitarse a ocultar su interfaz (NFR-R-01, NFR-R-03, NFR-P-02).
- **Aislamiento de fallos.** Los errores de un plugin o de un archivo incompatible se capturan y se informan sin interrumpir la sesión.

Cómo se verifica. Escenarios de Calidad 3 (errores en tiempo de ejecución), 4 (exportación), 7 (archivos corruptos o incompatibles) y 8 (consistencia entre datos y gráficas).

3.2.4. Usabilidad

Por qué importa. El diferenciador del producto, según el PMP, es ser la única herramienta gratuita con interfaz gráfica intuitiva que no requiere programar. Si el investigador necesita asistencia para operar el sistema, ese diferenciador se pierde. La usabilidad es, por tanto, el criterio con el que el Laboratorio validará el producto en la Fase 4.

Qué se exige. Toda operación que tarde más de 1 s debe mostrar progreso (NFR-U-03); los mensajes deben diferenciar información, advertencia y error (NFR-U-01); el resultado debe mostrarse en la misma pantalla en que se solicitó (NFR-U-02); cada control relevante debe tener una descripción emergente (NFR-U-07) e icono (NFR-U-04); las funcionalidades de edición deben concentrarse en *Home* (NFR-U-08); y el sistema debe dar la bienvenida a usuarios nuevos y mostrar una pantalla de inicio durante la carga (NFR-U-05, NFR-U-09).

Cómo lo resuelve la arquitectura.

- **Separación entre interfaz y cómputo.** Como el cálculo corre fuera del hilo de la interfaz, esta puede mostrar progreso y seguir respondiendo (NFR-U-03 se apoya en NFR-P-14).
- **Interfaz uniforme entre plugins.** Todos los módulos comparten el mismo patrón de panel de parámetros, de modo que aprender uno facilita usar los demás.
- **Retroalimentación centralizada.** La notificación de mensajes sigue un mecanismo común que distingue su severidad (NFR-U-01).

Cómo se verifica. Escenario de Calidad 5 (uso del sistema sin experiencia técnica) y las pruebas de usabilidad con el Laboratorio previstas en la Fase 4.

3.2.5. Soporte

Por qué importa. GammaLab v2.0 nace como una evolución de la versión 1.0 y se concibe para seguir creciendo: nuevos análisis, nuevos formatos y nuevas necesidades de otros laboratorios. El Soporte agrupa lo necesario para que ese crecimiento no degrade lo ya construido, y es el atributo que más interesa al equipo de desarrollo y a la comunidad que reutilice el código.

Qué se exige. Un nuevo plugin de análisis debe poder incorporarse registrándolo en el núcleo, sin modificar el núcleo ni los plugins existentes (NFR-S-05); ningún módulo debe alcanzar la estructura interna de los datos, sino usar una vía pública única para leer, modificar e invalidar los trials (NFR-S-02); los servicios compartidos de exportación, medición y ajustes deben ser entregados por el núcleo con una única instancia (NFR-S-03); la ejecución pesada debe pasar por un servicio de tareas (NFR-S-04); y el sistema debe poder instalarse con un ejecutable que registre la extensión de proyecto `.glab` (NFR-S-01).

Cómo lo resuelve la arquitectura.

- **Estilo microkernel.** Un núcleo mínimo de servicios y extensiones desacopladas descubiertas de forma dinámica, de modo que agregar capacidades no obliga a recompilar ni a tocar el núcleo.
- **Inversión de control.** Los servicios comunes se registran en el núcleo y se entregan a los plugins, en lugar de que cada plugin los construya o importe directamente.
- **Encapsulamiento del modelo de datos.** La entidad que representa la señal expone una interfaz pública para operar sobre sus trials y su frecuencia de muestreo, de modo que un cambio interno no rompe a los módulos que dependen de ella.

Cómo se verifica. Escenario de Calidad 1 (integración de un plugin externo).

3.2.6. Priorización y compromisos entre atributos

Los atributos no son independientes: mejorar uno suele costar algo de otro. Hacer explícitos estos compromisos evita decisiones accidentales y permite justificarlas ante los interesados. La Cuadro 6 recoge los más relevantes y la decisión adoptada en cada caso.

Cuadro 6: Compromisos entre atributos de calidad y decisión adoptada

Atributos en tensión	**Tensión**	**Decisión de la arquitectura**
Rendimiento frente a Confiabilidad	Submuestrear la gráfica acelera el dibujo, pero podría ocultar detalle de la señal.	El submuestreo se aplica únicamente a la representación visual y los datos permanecen intactos (NFR-P-15, NFR-R-05): se gana fluidez sin perder fidelidad en el análisis ni en la exportación.
Rendimiento frente a Confiabilidad	Ejecutar análisis en segundo plano introduce concurrencia y el riesgo de condiciones de carrera sobre los datos.	Se acepta la concurrencia a cambio de la fluidez, pero se compensa con un único escritor por dato (NFR-R-02) y con la terminación garantizada de los hilos (NFR-R-01).
Soporte frente a Rendimiento	Desacoplar los módulos mediante servicios y plugins añade capas de indirección.	Se prioriza el desacoplamiento, porque la indirección es pequeña frente al costo del cómputo científico; su efecto se vigila con las métricas de rendimiento (Cuadro 29).
Usabilidad frente a Rendimiento	Informar el progreso exige comunicación constante entre el hilo de cómputo y la interfaz.	El servicio de tareas incorpora la notificación de progreso como parte de su contrato (NFR-S-04), de modo que ningún plugin la implemente por su cuenta.

3.3. Supuestos

Un supuesto es una condición que se considera cierta durante el diseño sin que el sistema pueda garantizarla. Documentarlos es importante porque, si alguno deja de cumplirse, las decisiones que dependen de él deben revisarse. Por eso, la Cuadro 7 registra, para cada supuesto, su fuente y la consecuencia de que no se cumpla. Las restricciones que limitan el diseño, a diferencia de los supuestos, se establecen en la Sección 2.2 (*Limitaciones del sistema*).

Cuadro 7: Supuestos de GammaLab v2.0, su fuente y su impacto

ID	**Supuesto**	**Fuente**	**Impacto si no se cumple**
Entorno de ejecución			
A-01	El sistema se ejecuta sobre Windows 10 o superior, en arquitectura de 64 bits.	SRS	No se garantiza el instalador ni la portabilidad prevista (NFR-S-01).
A-02	El equipo cuenta con arquitectura multinúcleo (más de cuatro núcleos) para el procesamiento en segundo plano.	Equipo	La mejora de rendimiento esperada por la paralelización (20–30 %) puede no alcanzarse.

ID	Supuesto	Fuente	Impacto si no se cumple
A-03	El equipo dispone de recursos suficientes para manipular señales de gran tamaño, y los umbrales de rendimiento se miden sobre el hardware de referencia del laboratorio.	SRS, PMP	Los tiempos de carga, de graficación y de arranque (NFR-P-03, P-05, P-17) pueden superarse.
A-04	El usuario tiene permisos de lectura y escritura en el sistema de archivos local.	SRS	No es posible cargar señales, guardar proyectos ni exportar resultados.
A-05	Las dependencias científicas requeridas (por ejemplo, NumPy, SciPy, PyWavelets, PyABF y PyEDFlib) están disponibles en el entorno de ejecución, ya sea instaladas o empaquetadas en el ejecutable.	SRS	Los módulos de lectura de archivos y de análisis no pueden ejecutarse.
Datos de entrada			
A-06	Las señales a procesar ya fueron adquiridas: el sistema se dedica al posprocesamiento y no a la adquisición en tiempo real.	Sección 2.2	Requeriría un subsistema de adquisición y comunicación con hardware, hoy fuera de la arquitectura.
A-07	Los archivos de señal están en un formato soportado (`.abf`, `.edf`, `.bdf` o `.mat`).	SRS	El archivo se rechaza y se informa al usuario (Escenario de Calidad 7).
A-08	Los archivos `.mat` no incluyen la frecuencia de muestreo, por lo que el investigador la conoce y la ingresa manualmente antes de ejecutar análisis de frecuencia.	R80, CU-018	Los análisis de frecuencia se calcularían sobre un eje temporal incorrecto y producirían resultados erróneos.
A-09	Las señales usadas en PAC tienen al menos un trial válido; el PAC inter-canal y el PPC requieren al menos dos canales con trials generados.	SRS	El análisis no puede ejecutarse o carece de significado.
A-10	Las bandas de frecuencia seleccionadas están por debajo de la frecuencia de Nyquist de la señal.	SRS	El cálculo espectral no es válido para las bandas que la superen.
Usuarios y participación de los interesados			
A-11	El usuario tiene conocimientos básicos o intermedios de análisis de señales EEG (dominios del tiempo, frecuencia y tiempo-frecuencia), sabe interpretar gráficas y comodulogramas, y maneja inglés técnico básico.	SRS	La interfaz y la terminología deberían simplificarse y complementarse con mayor ayuda contextual (Escenario de Calidad 5).

ID	Supuesto	Fuente	Impacto si no se cumple
A-12	Los investigadores del Laboratorio participan activamente en las pruebas, la validación y la retroalimentación, y la institución facilita señales EEG para validar el sistema.	PMP	La validación de los resultados y del criterio de éxito de usuario final (Fase 4) quedaría sin sustento.
Arquitectura y extensiones			
A-13	La arquitectura microkernel heredada de la versión 1.0 es adecuada y escalable para incorporar las nuevas funcionalidades.	PMP	Sería necesario rediseñar el núcleo, con impacto en el alcance y el cronograma.
A-14	Los módulos desarrollados por terceros respetan la arquitectura modular y las interfaces definidas por GammaLab.	SRS	Un plugin defectuoso podría comprometer la estabilidad de la sesión (NFR-R-01, NFR-S-05).

4. Principios y decisiones arquitectónicas

La arquitectura de Gamma Lab 2.0 no parte de cero. Conserva el estilo microkernel de la versión 1.0, pero corrige las limitaciones identificadas en el diagnóstico de las secciones anteriores. Por esta razón, los principios y las decisiones de esta sección se formulan como respuesta a problemas concretos y no como aspiraciones generales. La siguente tabla sintetiza esos problemas y les asigna un identificador, de D1 a D15, que se utiliza en el resto de la sección para trazar cada principio y cada decisión hasta su origen.

ID	Problema
D1	El núcleo es un registro pasivo: no gestiona el ciclo de vida de las extensiones, `stop_plugins()` nunca se invoca y no es posible descargar ni recargar una extensión.
D2	El canal de eventos tiene una firma genérica (`str, object`), transporta solo dos eventos y la coordinación restante se resuelve con referencias directas entre componentes.
D3	`IPlugin` concentra procesamiento, interfaz gráfica y acceso a datos, por lo que el cálculo no puede probarse ni ejecutarse de forma aislada.
D4	Los servicios se obtienen con un *service locator* basado en cadenas literales: las dependencias quedan ocultas, no se verifican en la carga y un registro duplicado sobrescribe al anterior.
D5	El `DataStore` es un diccionario global y mutable sin un responsable de su ciclo de vida, por lo que la memoria crece de forma monótona durante la sesión.
D6	Los datos se replican: carga completa en `float64`, copia profunda hacia VTK, nuevas matrices en la caché de ensayos y remuestreo de canales EDF.
D7	La remoción de artefactos modifica los ensayos *in-place* desde un hilo secundario, rompe el encapsulamiento de `SignalDataset`, deja cachés obsoletas y no permite deshacer.
D8	Cada plugin conserva su ventana de renderizado VTK y sus tablas; al desactivarse no libera los recursos gráficos.

ID	Problema
D9	No existe un modelo de concurrencia: el procesamiento se ejecuta en el hilo de la interfaz y el usuario no distingue un cálculo en curso de un fallo.
D10	La concurrencia es una decisión local de cada plugin, con patrones distintos y sin un orquestador que limite las tareas simultáneas.
D11	La cancelación es parcial y defectuosa: bloquea la interfaz y el hilo puede seguir ejecutándose sobre objetos destruidos.
D12	No hay control de admisión: no se estima el costo en tiempo ni en memoria antes de ejecutar un proceso.
D13	Todo se ejecuta en un único proceso sin manejador global de excepciones: un error no capturado termina la aplicación y se pierde la sesión.
D14	Los errores se ocultan: el cargador omite los plugins que fallan al importarse y el núcleo mantiene registrados los que fallaron al inicializarse.
D15	No hay registro estructurado: la información se emite con `print`, sin niveles, sin marcas de tiempo y sin persistencia.

Cuadro 8: Síntesis del diagnóstico de Gamma Lab 1.0

4.1. Estilo arquitectónico: microkernel con núcleo activo

Gamma Lab 2.0 mantiene el estilo microkernel: un núcleo estable ofrece los servicios transversales y un conjunto de extensiones independientes implementa la funcionalidad de dominio, como el filtrado, la segmentación en ensayos, la FFT, la PSD, la transformada wavelet, el ERP y el PAC. El diagnóstico muestra que las limitaciones de la versión 1.0 no provienen del estilo, sino de su implementación: el núcleo existe estructuralmente, pero no gobierna el comportamiento del sistema (D1). Por ello, la decisión central de esta versión no es cambiar de estilo, sino asignar al núcleo las responsabilidades que el estilo le atribuye:

- Gobernar el ciclo de vida de las extensiones, desde su descubrimiento hasta su detención.
- Mediar la comunicación entre extensiones mediante eventos declarados.
- Centralizar la ejecución de los cómputos pesados fuera del hilo de la interfaz.
- Validar las extensiones y sus dependencias durante la carga.

Se consideraron dos alternativas. Reescribir el sistema como un monolito en capas habría simplificado el control del flujo, pero obligaba a reescribir las dieciséis extensiones existentes y sacrificaba la extensibilidad, que es una necesidad explícita del laboratorio. Un estilo de tuberías y filtros se ajusta bien al procesamiento de señales, pero no a una aplicación interactiva en la que el usuario navega entre módulos que comparten el mismo estado. El microkernel permite conservar las extensiones existentes y, al mismo tiempo, concentrar en el núcleo el control que hoy está disperso.

El núcleo no asume todas estas responsabilidades por sí mismo, sino que las distribuye entre un conjunto de servicios que no son reemplazables por extensiones:

- **Kernel**: mantiene el registro de extensiones y servicios, gobierna el ciclo de vida de las extensiones y ofrece el canal de eventos.

- **PluginManager**: descubre las extensiones a partir de su descriptor `properties.yml` y valida sus metadatos, la versión del contrato y sus dependencias antes de registrarlas.
- **TaskOrchestrator** (implementado como `TaskService`): encola y ejecuta los cómputos pesados fuera del hilo de la interfaz, informa el progreso, gestiona la cancelación y es el único componente que escribe en el estado compartido.
- **DataStore**: mantiene el estado compartido de la sesión, es decir, las señales, la señal activa, sus derivados y los resultados.
- **FileIOService**: adapta los formatos de archivo soportados al modelo de datos interno (`SignalDataset`).
- **ExportService**: exporta visualizaciones y resultados tabulares a formatos externos.
- **SettingsService**: gestiona la configuración persistente de la aplicación.
- **ProjectService**: persiste el espacio de trabajo en archivos `.glab` y es responsable del ciclo de vida del proyecto activo.

Alrededor del núcleo se organizan otras tres agrupaciones, coherentes con la vista lógica (Sección 5.2): la capa de presentación, construida con PyQt5 y VTK, que contiene la ventana principal y los elementos de interacción con las visualizaciones, como el menú contextual y el servicio de mediciones asociado a cada vista; las extensiones, que realizan el contrato `IPlugin`; y el modelo de datos, formado por `SignalDataset`, `TrialDataset` y la estructura del proyecto. La Figura 1 presenta esta organización.

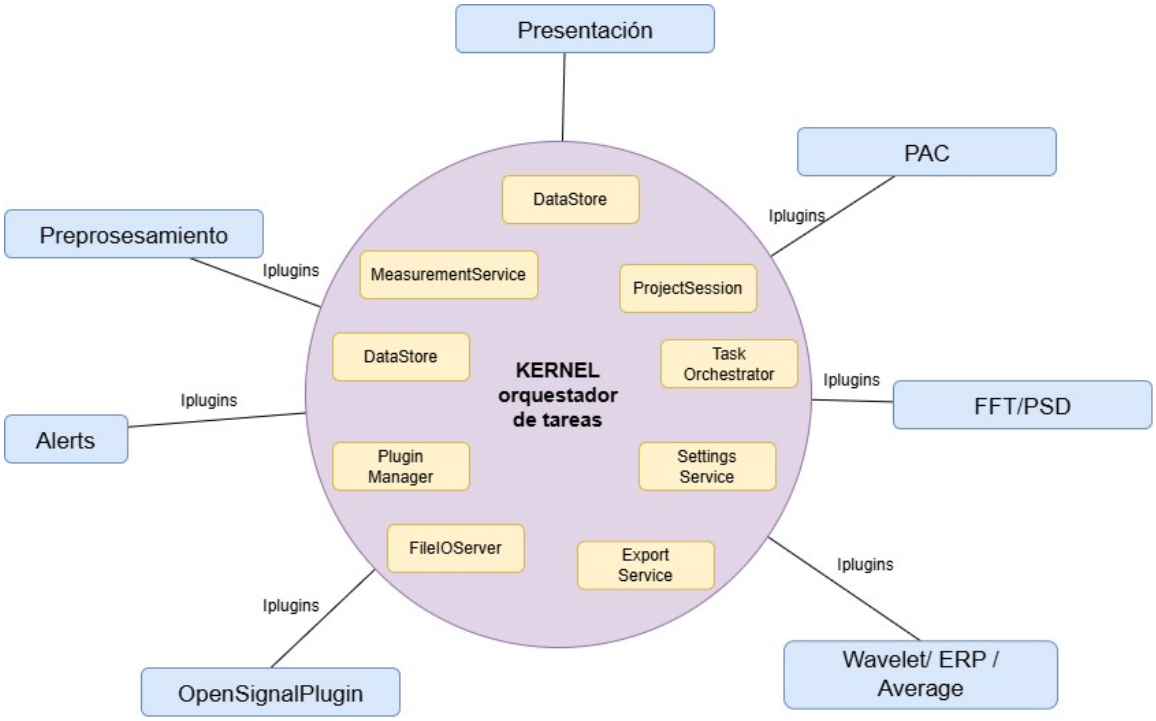


Figura 1: Arquitectura microkernel de Gamma Lab 2.0: el núcleo y sus servicios, la capa de presentación y las extensiones conectadas mediante el contrato `IPlugin`.

4.2. Principios arquitectónicos

Los principios se agrupan en las mismas cuatro áreas del diagnóstico. Cada uno se enuncia como una regla que el diseño debe cumplir e indica los problemas de la Tabla 8 a los que responde.

4.2.1. Núcleo y contratos

- **El núcleo gobierna el ciclo de vida de las extensiones:** Cada extensión pasa por estados explícitos: descubierta, inicializada, activa, inactiva, detenida o fallida. Solo el núcleo ejecuta las transiciones entre ellos. El cierre de la aplicación (CU-022) recorre las extensiones e invoca su detención, y una extensión cuya inicialización falla queda en estado fallida y visible para el usuario, en lugar de permanecer registrada como si funcionara. *Responde a: D1, D14.*
- **Las extensiones se comunican mediante eventos declarados:** Las extensiones no se referencian entre sí. Se coordinan a través de un catálogo cerrado de eventos del núcleo, por ejemplo, señal cargada, ensayos generados, conjunto derivado creado o tarea finalizada, y cada evento tiene una carga útil de estructura definida. Un evento que no pertenece al catálogo se rechaza al emitirse. Así, el canal de eventos deja de tener un papel marginal y se convierte en el mecanismo de desacoplamiento que el estilo prevé. *Responde a: D2.*
- **El cómputo se separa de la presentación:** El cálculo de cada extensión se implementa como una función sin dependencias de Qt ni de VTK, que recibe arreglos y parámetros y retorna un resultado. La extensión conserva la construcción de su vista y la solicitud de ejecución al orquestador. Esta separación permite probar el cálculo de forma aislada contra las referencias de MATLAB y ejecutarlo en un hilo o en un proceso trabajador sin arrastrar objetos de presentación. *Responde a: D3.*
- **Las dependencias se declaran y se verifican durante la carga:** Cada extensión declara en su `properties.yml` la versión del contrato `IPlugin` con la que fue escrita y los servicios que requiere. Durante el arranque, el PluginManager verifica que la versión sea compatible, que los servicios existan y que el identificador no esté repetido. Si alguna condición no se cumple, la extensión no se carga y el usuario recibe la causa. Las claves de los servicios se definen en un único lugar, en vez de repetirse como cadenas literales. El *service locator* se conserva, pero sus errores pasan del tiempo de ejecución al arranque. *Responde a: D4, D14.*

4.2.2. Estado y memoria

- **El estado tiene un dueño y los datos originales no se modifican:** El `DataStore` tiene un único escritor, el TaskOrchestrator (R54): las extensiones lo consultan, pero no lo modifican directamente. Las transformaciones de preprocesamiento no alteran la señal ni los ensayos de origen, sino que producen conjuntos derivados que referencian a su fuente. Para que esta regla no multiplique el consumo de memoria, un derivado materializa solo lo que cambia; por ejemplo, el descarte de ensayos se representa con una máscara de índices y no con una nueva matriz. Cada conjunto tiene una versión, y las cachés que dependen de él se invalidan cuando esta cambia. Por último, el ProjectService es el responsable del ciclo de vida del proyecto, de modo que al cerrarlo o reemplazarlo (CU-021, CU-003) se liberan la señal y todos sus derivados. *Responde a: D5, D7.*
- **Ningún componente copia la señal sin necesidad:** La visualización trabaja sobre una representación submuestreada de la señal (CU-019) y no sobre una copia profunda

completa. La selección de ensayos se resuelve con índices o vistas de NumPy. El tipo numérico se elige por etapa: se usa precisión simple donde la diferencia frente a las referencias de MATLAB está dentro de la tolerancia aceptada y precisión doble donde no lo está. *Responde a: D6.*

- **Los recursos se liberan de forma determinista:** Al desactivarse una extensión, por ejemplo al cambiar de módulo (CU-008), esta libera su ventana de renderizado, sus observadores y sus tablas de VTK, y su vista se reconstruye bajo demanda cuando el usuario regresa. Al cerrar el proyecto o la aplicación (CU-021, CU-022) no quedan hilos, procesos ni contextos gráficos activos. *Responde a: D5, D8.*

4.2.3. Concurrencia

- **La ejecución de los cómputos pesados está centralizada y es cancelable:** Ninguna extensión crea hilos ni procesos. Toda operación de cómputo pesado se solicita al TaskOrchestrator (se considera pesada la que, con los datos típicos del laboratorio, puede tardar más de 1 segundo en el hardware de referencia), que la encola, la ejecuta fuera del hilo de la interfaz y de a una a la vez (R70), presenta un indicador de progreso cuando supera un segundo y permite cancelarla (CU-014). La cancelación es cooperativa: el cómputo consulta periódicamente una señal de cancelación. Si no responde dentro de un tiempo acotado, el orquestador fuerza su terminación. *Responde a: D9, D10, D11.*

- **Antes de ejecutar se estima el costo:** El orquestador estima la memoria que requiere un cómputo a partir de las dimensiones de la entrada y de sus parámetros, por ejemplo, el número de canales, muestras y frecuencias de una transformada wavelet o de un análisis PAC, y la compara con la memoria disponible. Si la estimación supera lo disponible, el sistema advierte al usuario y le propone reducir los parámetros; si la ejecución es claramente inviable, la rechaza antes de iniciarla. *Responde a: D12.*

4.2.4. Tolerancia a fallos y observabilidad

- **Los fallos se contienen y se hacen visibles:** Un fallo no debe terminar la aplicación ni pasar inadvertido. Un manejador global captura las excepciones no controladas del hilo principal y de los *slots* de Qt. Las excepciones de un cómputo se capturan en el trabajador y llegan a la interfaz como un resultado fallido. Los cómputos con riesgo de agotar la memoria o de fallar en código nativo se ejecutan en un proceso aislado, de modo que su caída no arrastra a la aplicación. En todos los casos el usuario recibe un mensaje comprensible y la sesión se conserva. *Responde a: D13, D14.*

- **El sistema es observable:** Todos los componentes registran sus eventos con nivel de severidad, marca de tiempo, componente de origen e identificador de la tarea cuando aplica. El registro se persiste en un archivo rotativo en el directorio del usuario, por lo que sobrevive a un cierre inesperado y permite reconstruir qué operación se ejecutaba cuando ocurrió un fallo. *Responde a: D15.*

4.3. Decisiones arquitectónicas

La Tabla 9 resume las decisiones arquitectónicas que materializan los principios anteriores. Para cada una se indica el problema del diagnóstico al que responde, la alternativa que se descartó, sus consecuencias y su estado: *implementada*, si ya está presente en el código; *parcial*, si existe una parte; o *planificada*, si debe construirse en esta versión.

ID	Decisión	D	Alternativa descartada	Consecuencias
ADR-01	Mantener el estilo microkernel y asignar al núcleo responsabilidades propias.	D1	Monolito en capas; tuberías y filtros.	Se conservan las extensiones y el contrato existentes. El núcleo pasa a ser un componente crítico que requiere pruebas propias.
ADR-02	Ciclo de vida de las extensiones como máquina de estados gobernada por el Kernel.	D1, D14	Que cada extensión gestione su propia activación, como en 1.0.	Cierre ordenado y extensiones fallidas visibles. Exige que toda extensión implemente una detención completa.
ADR-03	Catálogo de eventos con carga útil tipada.	D2	Canal genérico (`str, object`) y referencias directas.	Los errores de tópico se detectan al emitir. El catálogo se convierte en un contrato que debe versionarse.
ADR-04	Separar el cómputo de cada extensión en un módulo `compute` sin dependencias de Qt ni VTK.	D3	`IPlugin` único con lógica, vista y datos.	El cálculo es comprobable y ejecutable en un trabajador. Cada extensión pasa a tener dos módulos.
ADR-05	Descubrimiento declarativo validado: `properties.yml` con versión del contrato, dependencias y unicidad.	D4, D14	Contenedor de inyección de dependencias, por el costo de migrar todas las extensiones.	Los errores de configuración se detectan en el arranque. Se amplía el esquema del descriptor.
ADR-06	`DataStore` con escritor único y conjuntos derivados inmutables.	D5, D7	Escritura libre protegida con exclusión mutua; modificación *in-place*.	Estado consistente, trazable y con posibilidad de deshacer. Los derivados deben materializar solo lo que cambia.
ADR-07	Estrategia de memoria: representación submuestreada, sin copia profunda hacia VTK y tipo numérico por etapa.	D6	Carga perezosa o memoria mapeada, que depende del formato de cada archivo.	Menor consumo de memoria. La precisión debe validarse contra las referencias de MATLAB.
ADR-08	Liberación determinista de recursos al desactivar una extensión, cambiar de módulo y cerrar.	D5, D8	Conservar en caché todas las vistas instanciadas.	Consumo acotado de memoria de video y de sistema. Reconstruir una vista al regresar tiene un costo de tiempo.

Continúa en la siguiente página

ID	Decisión	D	Alternativa descartada	Consecuencias
ADR-09	TaskOrchestrator híbrido: hilos por defecto y proceso trabajador para los cómputos aislables (Sección 4.3.1).	D9–D11, D13	Solo hilos; solo multiprocesamiento.	Interfaz responsiva, cancelación forzada real y aislamiento de fallos. Costo de serializar arreglos grandes.
ADR-10	Control de admisión mediante la estimación del costo de cada cómputo.	D12	Ejecutar sin estimar, como en 1.0.	Se evitan congelamientos y cálculos perdidos. Cada extensión debe proveer su estimación.
ADR-11	Manejador global de excepciones y captura en los trabajadores con notificación a la interfaz.	D13, D14	Manejo local en cada extensión.	La sesión se conserva ante fallos. Los mensajes deben ser comprensibles para el usuario.
ADR-12	Registro estructurado con el módulo `logging` y archivo rotativo.	D15	Mensajes con `print` en consola.	Diagnóstico posterior a un fallo sin reproducirlo. Deben reemplazarse las llamadas existentes a `print`.
ADR-13	Python, PyQt5 y VTK como base tecnológica.	—	Migrar a PySide6.	Base madura para cálculo y visualización científica. La licencia GPL de PyQt5 condiciona la distribución, y VTK solo puede renderizar en el hilo principal.

Cuadro 9: Decisiones arquitectónicas de Gamma Lab 2.0

4.3.1. Modelo de concurrencia del TaskOrchestrator

La decisión ADR-09 merece una explicación adicional, porque es la que más condiciona la respuesta del sistema. El `TaskService` actual ya resuelve buena parte de D9 y D10: mantiene una cola de tareas, ejecuta una sola a la vez en un hilo trabajador, descarta las tareas pendientes de una extensión cuando esta solicita una nueva y ofrece a cada cómputo una señal de cancelación cooperativa. Cuando el usuario cancela una tarea que no responde en dos segundos, el servicio deja de esperarla y descarta su resultado, pero no puede detenerla. Esta limitación es propia de los hilos de Python, que no pueden terminarse de forma segura desde fuera: el hilo desacoplado sigue consumiendo CPU y memoria hasta terminar, y un `MemoryError` o un error nativo dentro de él sigue afectando a toda la aplicación.

Por ello, la terminación forzada que exige CU-014 y el aislamiento de fallos de P10 solo pueden lograrse con un proceso trabajador, que sí puede terminarse sin comprometer el proceso principal.

Se adopta un modelo híbrido:

- **Hilo trabajador** para los cómputos breves o de bajo consumo de memoria, como el filtrado, la segmentación o el promedio de ensayos, donde el costo de crear un proceso y de transferir los datos superaría el beneficio.
- **Proceso trabajador** para los cómputos intensivos y aislables, como el PAC, la transformada wavelet y el índice de modulación, cuya duración y consumo de memoria justifican poder terminarlos y aislar su posible falla.

El modelo tiene dos costos que el diseño debe atender. El primero es la transferencia de arreglos grandes entre procesos, que se mitiga con memoria compartida en lugar de serialización completa y con un proceso trabajador persistente que evita el costo de arranque en cada tarea. El segundo es que el resultado siempre debe volver al hilo principal, porque VTK solo puede renderizar en él. En ambos casos, la extensión no conoce el mecanismo utilizado: entrega una función de cómputo (P3) y el orquestador decide cómo ejecutarla.

4.4. Patrones aplicados

La Tabla anterior resume los patrones que materializan los principios y las decisiones anteriores.

Patrón	**Problema que resuelve**	**Aplicación en Gamma Lab 2.0**
Microkernel / Plugin	Agregar capacidades sin modificar el núcleo.	Cada extensión declara un `properties.yml` y una clase que realiza `IPlugin`; el PluginManager la descubre y la registra en el Kernel.
State	Gobernar el ciclo de vida de las extensiones (D1).	El Kernel mantiene el estado de cada extensión y solo permite las transiciones definidas (P1, ADR-02).
Service Locator	Ofrecer servicios compartidos sin que las extensiones los instancien (D4).	El Kernel expone los servicios por clave. Sus dependencias se declaran y se verifican en la carga para mitigar sus riesgos (P4, ADR-05).
Observer	Comunicar cambios sin referencias directas (D2).	El canal de eventos del Kernel, basado en señales de Qt, notifica a los suscriptores los eventos del catálogo (P2, ADR-03).
Command con señal de cancelación	Ejecutar, seguir y cancelar cómputos de forma uniforme (D10, D11).	Cada cómputo se encapsula como una tarea con su `TaskHandle` (progreso, finalización, fallo, cancelación) y su `TaskContext` (señal de cancelación cooperativa).
Single Writer	Mantener la consistencia del estado compartido (D5, D7).	Solo el TaskOrchestrator escribe en el `DataStore` (P5, ADR-06).
Adapter	Integrar representaciones incompatibles.	El `FileIOService` adapta los formatos ABF, EDF y MAT a `SignalDataset`; los adaptadores de VTK convierten arreglos de NumPy en estructuras de visualización.
Strategy	Intercambiar extensiones detrás de un mismo contrato.	El núcleo y la ventana principal operan sobre `IPlugin` sin conocer la extensión concreta.
Template Method	Reutilizar el procedimiento común de los análisis de acoplamiento.	`PACAnalysisPlugin` define el cálculo del mapa de fase y frecuencia, y sus especializaciones de uno y dos canales completan los pasos propios.

Cuadro 10: Patrones arquitectónicos aplicados en Gamma Lab 2.0.

4.5. Trazabilidad entre el diagnóstico y las decisiones

La Tabla 11 cierra la sección relacionando cada problema del diagnóstico con el principio y la decisión que lo atienden, y con los casos de uso de la vista de escenarios y los escenarios de calidad en los que se verifica su solución.

D	Principio	Decisión	Verificación (caso de uso / escenario)
D1	P1	ADR-01, ADR-02	CU-011, CU-022
D2	P2	ADR-03	CU-003, CU-004
D3	P3	ADR-04	Escenario 1; métrica de mantenibilidad (cobertura)
D4	P4	ADR-05	CU-011; Escenario 1
D5	P5, P7	ADR-06, ADR-08	CU-003, CU-021; Escenario 6
D6	P6	ADR-07	CU-019; Escenarios 2 y 6
D7	P5	ADR-06	CU-007, CU-016; Escenario 8
D8	P7	ADR-08	CU-008, CU-022
D9	P8	ADR-09	CU-020; Escenario 2
D10	P8	ADR-09	CU-020
D11	P8	ADR-09	CU-014; métrica de robustez ante cancelación
D12	P9	ADR-10	CU-020; Escenario 2
D13	P10	ADR-09, ADR-11	Escenarios 3 y 7
D14	P1, P4, P10	ADR-02, ADR-05, ADR-11	CU-011; Escenarios 1 y 3
D15	P11	ADR-12	Escenario 3

Cuadro 11: Trazabilidad entre el diagnóstico, los principios y las decisiones

5. Vistas Arquitectónicas (4+1)

5.1. Vista de Escenarios

5.1.1. Propósito

El diagrama de casos de uso describe el comportamiento observable de Gamma Lab 2.0 desde la perspectiva de su usuario. Representa las funcionalidades que el sistema ofrece, el actor que las utiliza y las relaciones de dependencia, extensión y especialización entre ellas. Cada caso de uso agrupa uno o más requerimientos funcionales, por lo que el modelo vincula la especificación de requerimientos con la estructura de la arquitectura. Además, al organizar los casos de uso en subsistemas, el diagrama anticipa la descomposición funcional del sistema y muestra cómo se distribuyen las responsabilidades entre sus componentes.

5.1.2. Notación

El diagrama sigue la notación de casos de uso definida en la especificación UML 2.5.1 del Object Management Group [17]. El sistema se representa como un contenedor general con el estereotipo «system». Dentro de él, los casos de uso se agrupan en paquetes con el estereotipo «subsistema», cada uno asociado a un área de responsabilidad funcional. Las relaciones entre casos de uso son de tres tipos:

- **Inclusión («include»):** línea punteada que va del caso base al caso incluido. Indica que el comportamiento incluido es obligatorio y forma parte de la ejecución del caso base.
- **Extensión («extend»):** línea punteada que va del caso extensor al caso base. Indica un comportamiento opcional o condicional que amplía el flujo del caso base sin modificarlo.
- **Generalización:** línea continua que va del caso especializado al caso general. Indica que el caso hijo hereda y especializa el comportamiento del padre.

Los casos de uso marcados con el estereotipo «abstract» no se ejecutan directamente. Su función es definir un comportamiento común que concretan sus especializaciones.

5.1.3. Visión general del modelo

El diagrama sigue el flujo de interacción de Gamma Lab 2.0, desde que el usuario inicia la aplicación hasta que obtiene sus resultados. El sistema tiene un único actor, el Investigador, que interactúa exclusivamente con la capa de presentación. Esta capa es el único punto de entrada al sistema y concentra la navegación entre módulos y la visualización de las señales. Después de cargar una señal o abrir un proyecto, el investigador puede acondicionarla con las operaciones de preprocesamiento y luego aplicarle los análisis disponibles. La interfaz no ejecuta directamente ninguna de estas operaciones, sino que las delega en el orquestador del núcleo. Este componente actúa como mediador entre la presentación y la lógica de procesamiento: recibe las solicitudes de procesamiento pesado, administra su ciclo de ejecución y permite cancelarlas. Gracias a esta mediación, la interfaz queda desacoplada de los algoritmos de procesamiento, cada subsistema conserva una responsabilidad bien definida y el control de la ejecución queda centralizado en el núcleo, como establece el estilo arquitectónico de microkernel. Al final del flujo, los resultados se persisten o se exportan, y el proyecto se cierra junto con la aplicación. Este recorrido se refleja en la organización del modelo en cinco subsistemas: Interfaz y Usabilidad, Núcleo y Ciclo de Vida, Preprocesamiento, Análisis y Exportación.

Las secciones siguientes describen cada uno. La Figura 2 presenta el diagrama de casos de uso del sistema.

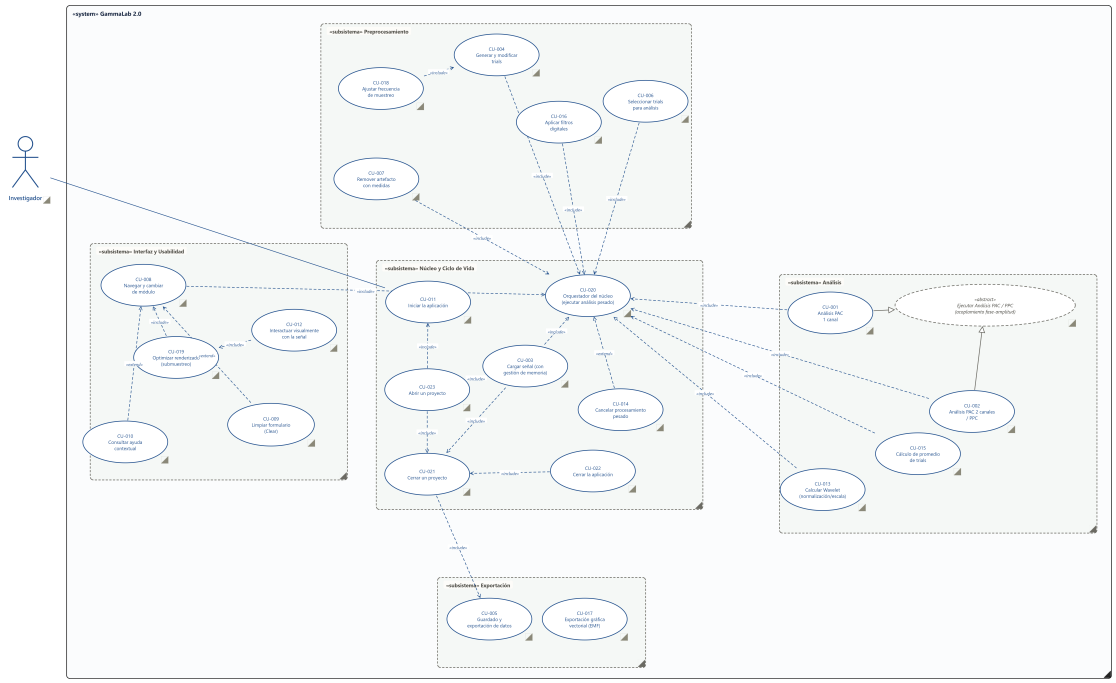


Figura 2: Diagrama de casos de uso del sistema completo Gamma Lab 2.0 (23 CU en 5 subsistemas). Las relaciones *include* / *extend* se tomaron literalmente del campo "Extensiones" de cada ficha; ninguna es inferida por similitud temática.

La Figura 2 revela que el subsistema **Núcleo y Ciclo de Vida** — en particular CU-020, el orquestador de tareas — es el verdadero centro de gravedad arquitectónico: diez de los otros veintidós casos de uso lo incluyen explícitamente. Esa centralidad se concreta a continuación en tres escenarios representativos.

5.1.4. Subsistema Interfaz y Usabilidad

El subsistema **Interfaz y Usabilidad** agrupa los casos de uso que definen cómo interactúa el Investigador con Gamma Lab 2.0 a través de la capa de presentación. Su responsabilidad no es procesar señales, sino garantizar que la interacción sea comprensible, fluida y estable en el uso de recursos. Por eso concentra principalmente requerimientos no funcionales de usabilidad, rendimiento y confiabilidad, junto con algunos requerimientos funcionales de visualización. El Cuadro 12 resume los casos de uso que lo componen.

Cuadro 12: Casos de uso del subsistema Interfaz y Usabilidad

Caso de uso	Nombre	Descripción
CU-008	Navegar y cambiar de módulo	Permite desplazarse entre los módulos funcionales del sistema, liberando los recursos del módulo saliente y conservando la señal activa en memoria.
CU-009	Limpiar formulario (Clear)	Restablece a sus valores por defecto los parámetros de una funcionalidad, sin modificar los resultados ya presentados.
CU-010	Consultar ayuda contextual	Muestra una descripción emergente del propósito de cada control de la interfaz.
CU-012	Interactuar visualmente con la señal	Permite explorar señales y resultados mediante zoom, lectura de coordenadas y vista Butterfly, sin alterar los datos.
CU-019	Optimizar renderizado (submuestreo)	Reduce la densidad de la representación gráfica para mantener la fluidez del renderizado, sin modificar los datos en memoria.

El subsistema se organiza alrededor de CU-008, que funciona como eje de la interacción y como único punto de conexión con el subsistema Núcleo y Ciclo de Vida. Esta posición se debe a que el cambio de módulo es una transición en el ciclo de vida de las extensiones. Al desactivarse, cada módulo debe liberar sus recursos gráficos y sus detectores de eventos de forma determinista. Al activarse, el módulo destino debe reutilizar los componentes ya construidos, sin recargar la señal desde disco. Los demás casos de uso del subsistema se vinculan con este eje. CU-010 lo extiende como un comportamiento opcional, activado por el usuario, que no altera el estado del sistema. CU-019 se relaciona con la navegación porque la optimización del renderizado condiciona la fluidez de cualquier vista presentada.

El diagrama también muestra un segundo agrupamiento, formado por CU-012 y CU-019. CU-012 incluye a CU-019, lo que expresa que toda interacción visual sobre una señal pasa obligatoriamente por la estrategia de submuestreo de representación. Esta relación materializa la separación entre el modelo de datos y la vista: el zoom, el desplazamiento y la lectura de coordenadas modifican solo la representación, mientras que los datos en memoria permanecen intactos.

Por último, CU-009 aparece en el diagrama vinculado con CU-019 mediante una relación de extensión. Este caso de uso representa una operación sobre el estado del formulario de cada funcionalidad: restablece los parámetros a sus valores por defecto sin modificar los resultados ya presentados ni los datos en memoria. Su ejecución está condicionada a que no exista un cómputo en curso lanzado desde ese formulario, lo que mantiene la coherencia entre los parámetros mostrados y la tarea en ejecución.

En conjunto, el subsistema resuelve dentro de la capa de presentación los atributos de calidad más sensibles para el usuario: la capacidad de respuesta, la fluidez del renderizado y el consumo acotado de memoria y recursos gráficos. Al concentrar su conexión con el resto del sistema en CU-008, mantiene la lógica de interacción desacoplada de la lógica de procesamiento y delega la ejecución de las operaciones pesadas en el orquestador del núcleo.

5.1.5. Subsistema Núcleo y Ciclo de Vida

El subsistema **Núcleo y Ciclo de Vida** materializa el estilo arquitectónico de microkernel. En él se registran los servicios compartidos, se descubren las extensiones y se centraliza la ejecución

de las operaciones pesadas, por lo que concentra requerimientos de rendimiento, confiabilidad y soporte. El Cuadro 13 resume los casos de uso que lo componen.

Cuadro 13: Casos de uso del subsistema Núcleo y Ciclo de Vida

Caso de uso	Nombre	Descripción
CU-011	Iniciar la aplicación	Arranca el sistema, registra en el núcleo los servicios compartidos y descubre las extensiones disponibles.
CU-022	Cerrar la aplicación	Finaliza la ejecución liberando hilos, extensiones, recursos gráficos y memoria.
CU-023	Abrir un proyecto	Restaura un proyecto guardado, con su señal de origen y el trabajo registrado sobre ella.
CU-003	Cargar señal (con gestión de memoria)	Carga un archivo de señal como nuevo proyecto activo, liberando previamente la memoria del proyecto anterior.
CU-021	Cerrar un proyecto	Cierra el proyecto activo, ofrece guardarlo y libera la memoria asociada a la señal y sus derivados.
CU-020	Orquestador del núcleo (ejecutar análisis pesado)	Recibe, encola y ejecuta fuera del hilo de la interfaz las operaciones de cómputo pesado y publica sus resultados en el estado compartido.
CU-014	Cancelar procesamiento pesado	Interrumpe una tarea en curso, libera su hilo trabajador y descarta el resultado parcial.

El diagrama organiza estos casos de uso en tres agrupamientos que corresponden a tres ciclos de vida distintos. El primero es el ciclo de vida de la aplicación, formado por CU-011 y CU-022. CU-011 deja disponibles los servicios del núcleo mediante inversión de control, de modo que ninguna extensión los instancia por su cuenta, y registra las extensiones sin que sea necesario modificar el núcleo ni las ya existentes. CU-022 es su contraparte: garantiza que al terminar la ejecución no queden hilos, procesos ni contextos gráficos activos.

El segundo agrupamiento corresponde al ciclo de vida del proyecto y lo forman CU-023, CU-003 y CU-021. En el diagrama, CU-021 recibe relaciones de inclusión desde CU-023, CU-003 y CU-022. Esto lo convierte en el punto único de cierre del proyecto: toda operación que reemplace o termine el proyecto activo reutiliza el mismo comportamiento para ofrecer el guardado y liberar la memoria. A su vez, CU-023 incluye a CU-011 y a CU-003, lo que expresa que abrir un proyecto puede implicar iniciar la aplicación y que la restauración de la señal se apoya en el mismo mecanismo de carga. CU-021 también se relaciona con el subsistema Exportación mediante la inclusión de CU-005, del que obtiene la persistencia del proyecto.

El tercer agrupamiento corresponde al ciclo de vida de las tareas de cómputo y lo forman CU-020 y CU-014. CU-020 es el elemento central del subsistema y del modelo completo, ya que recibe las relaciones de inclusión de los casos de uso de Preprocesamiento y Análisis, además de la de CU-003. Su función es desacoplar a las extensiones de la gestión de la concurrencia. Las extensiones no administran hilos propios, sino que solicitan la ejecución al núcleo. El orquestador encola las solicitudes y las ejecuta en hilos trabajadores, mantiene la interfaz responsiva durante el cómputo y publica los resultados en el estado compartido mediante escritura sincronizada con un único propietario. CU-014 extiende a CU-020 como un comportamiento opcional, activado por el usuario, que propaga la cancelación a la tarea en ejecución y garantiza la liberación de su hilo trabajador, aunque el cómputo no haya terminado.

En conjunto, el subsistema concentra las responsabilidades transversales del sistema: la administración de servicios y extensiones, la gestión de memoria entre proyectos y el control de la

ejecución concurrente. Al centralizar estas responsabilidades en el núcleo, los demás subsistemas pueden limitarse a su lógica funcional, y la incorporación de nuevas extensiones se reduce a registrarlas en el núcleo y conectarlas al orquestador.

5.1.6. Subsistema Preprocesamiento

El subsistema **Preprocesamiento** agrupa las operaciones que acondicionan la señal antes de su análisis. Su responsabilidad es transformar la señal y sus segmentaciones sin comprometer la integridad de los datos originales, por lo que concentra requerimientos funcionales de transformación y requerimientos de soporte orientados al encapsulamiento del modelo de datos. El Cuadro 14 resume los casos de uso que lo componen.

Cuadro 14: Casos de uso del subsistema Preprocesamiento

Caso de uso	Nombre	Descripción
CU-004	Generar y modificar trials	Segmenta la señal en ensayos a partir de un canal de estímulo y permite editar o descartar trials ya generados.
CU-018	Ajustar frecuencia de muestreo	Corrige la frecuencia de muestreo de la señal activa y recalcula su eje temporal.
CU-016	Aplicar filtros digitales	Aplica un filtro Butterworth pasa-banda de fase cero y registra el resultado como una señal derivada.
CU-007	Remover artefacto con medidas	Elimina artefactos de estimulación en los trials, mostrando antes las medidas del segmento afectado.
CU-006	Seleccionar trials para análisis	Define el subconjunto de trials que participa en un análisis y deja constancia de cuáles se usaron.

En el diagrama, todos los casos de uso del subsistema, con excepción de CU-018, mantienen una relación de inclusión con el orquestador del núcleo (CU-020). Esto expresa que ninguna transformación se ejecuta en el hilo de la interfaz: el subsistema define qué operación se aplica sobre la señal, y el núcleo determina cómo y cuándo se ejecuta. Así, las extensiones de preprocesamiento contienen solo su lógica de transformación y quedan desacopladas de la gestión de la concurrencia.

La única relación interna del subsistema es la inclusión de CU-004 por parte de CU-018. Como los trials se segmentan sobre el eje temporal de la señal, un cambio en la frecuencia de muestreo deja desactualizadas las segmentaciones existentes, y el sistema ofrece regenerarlas con el mismo mecanismo de corte. Por esta vía, CU-018 llega al orquestador de forma indirecta, a través de CU-004.

Más allá de las relaciones representadas, los casos de uso del subsistema comparten un mismo principio de diseño: los datos originales no se modifican. Cada transformación se registra como un conjunto derivado de la señal o de los trials, y el acceso al modelo se hace siempre a través de su API pública, sin que ninguna extensión manipule su estructura interna. Cuando un cambio afecta a datos ya representados o analizados, el sistema invalida las cachés dependientes y advierte qué resultados quedaron desactualizados. Con este principio se conserva la trazabilidad entre la señal original y cada una de sus transformaciones, y se reduce el acoplamiento entre las extensiones y el modelo de datos.

En conjunto, el subsistema actúa como la etapa de acondicionamiento previa al análisis. Combina

la delegación de la ejecución en el núcleo con la preservación de los datos originales, de modo que el investigador puede probar distintas transformaciones sin perder la señal de partida.

5.1.7. Subsistema Análisis

El subsistema **Análisis** agrupa los cómputos que extraen información de la señal acondicionada, como el acoplamiento entre bandas de frecuencia, la representación tiempo-frecuencia y el promedio de ensayos. Concentra requerimientos funcionales de cálculo y visualización de resultados, junto con requerimientos de rendimiento que acotan el tiempo de los cómputos más costosos. El Cuadro 15 resume los casos de uso que lo componen.

Cuadro 15: Casos de uso del subsistema Análisis

Caso de uso	Nombre	Descripción
CU-001	Análisis PAC de un canal	Calcula el acoplamiento fase-amplitud entre dos bandas de frecuencia de un mismo canal, sobre un trial único o el promedio de trials.
CU-002	Análisis PAC de dos canales / PPC	Calcula el acoplamiento fase-amplitud entre dos canales distintos, o la coherencia de fase entre ellos.
CU-013	Calcular Wavelet	Calcula la transformada wavelet de la señal con normalización y escala configurables, y presenta el escalograma tiempo-frecuencia.
CU-015	Cálculo de promedio de trials	Promedia todos los trials o un subconjunto seleccionado para reducir el ruido de la señal.

En el diagrama, todos los casos de uso concretos del subsistema mantienen una relación de inclusión con el orquestador del núcleo (CU-020). Esto es consecuente con la naturaleza del subsistema, que reúne los cómputos más costosos del sistema. Delegar su ejecución en el núcleo permite que la interfaz siga respondiendo durante el cálculo, que el usuario pueda cancelarlo y que los resultados se publiquen en el estado compartido de forma sincronizada. Cada extensión de análisis queda así limitada a su lógica de cálculo, sin intervenir en la gestión de la concurrencia.

La relación interna del subsistema es una generalización: CU-001 y CU-002 especializan el caso de uso abstracto Ejecutar análisis PAC / PPC, que define el procedimiento común de cálculo del acoplamiento fase-amplitud y sus visualizaciones. Las especializaciones se distinguen por el origen de los datos. CU-001 toma la fase y la amplitud de un mismo canal, y CU-002 las toma de canales distintos y añade la coherencia de fase. Esta estructura permite reutilizar el comportamiento común y agregar nuevas variantes de acoplamiento sin modificar las existentes.

CU-013 y CU-015 no participan en esta jerarquía y se representan como análisis independientes. CU-013 produce la representación tiempo-frecuencia de la señal y tiene un tiempo máximo de cómputo definido. CU-015 produce la serie promediada de los trials y opera sobre el subconjunto seleccionado en el subsistema Preprocesamiento.

En conjunto, el subsistema consume los datos acondicionados por el preprocesamiento y produce los resultados que después se visualizan y exportan. Al delegar la ejecución en el núcleo y organizar los análisis de acoplamiento mediante generalización, el subsistema puede crecer con nuevos análisis sin afectar la estabilidad de la interfaz ni la estructura de los análisis existentes.

5.1.8. Subsistema Exportación

El subsistema **Exportación** agrupa los casos de uso que trasladan el trabajo realizado en Gamma Lab 2.0 al sistema de archivos local. Su responsabilidad es la persistencia del proyecto y la exportación de resultados numéricos y gráficos, por lo que concentra requerimientos funcionales de formato y nomenclatura de archivos, junto con requerimientos de rendimiento que acotan el tiempo de cada exportación. El Cuadro 16 resume los casos de uso que lo componen.

Cuadro 16: Casos de uso del subsistema Exportación

Caso de uso	Nombre	Descripción
CU-005	Guardado y exportación de datos	Guarda el proyecto en formato .glab o exporta los resultados del análisis activo en formatos tabulares (Excel, CSV, JSON) o de imagen rasterizada (JPEG, PNG).
CU-017	Exportación gráfica vectorial (EMF)	Exporta las visualizaciones de los análisis en formato vectorial, conservando sus ejes y escalas para uso en publicaciones.

En el diagrama, el subsistema se conecta con el resto del sistema mediante una única relación: la inclusión de CU-005 por parte de CU-021, Cerrar un proyecto, del subsistema Núcleo y Ciclo de Vida. Esta relación expresa que el guardado del proyecto no es una operación aislada, sino una parte obligatoria del ciclo de vida del proyecto: toda operación que cierre o reemplace el proyecto activo pasa por el mismo mecanismo de persistencia. El archivo de proyecto almacena una referencia a la señal de origen junto con los trials, los análisis y sus parámetros, sin duplicar los datos de la señal. Así, el proyecto se mantiene liviano y puede restaurarse a partir del archivo original.

CU-017 se representa sin relaciones con otros casos de uso, porque es una operación que el usuario invoca directamente sobre una visualización ya generada. Complementa a CU-005 con un formato vectorial que conserva cada elemento gráfico por separado, en lugar de una imagen plana. Los dos casos de uso comparten las mismas convenciones de salida: el nombre sugerido del archivo incluye el nombre de la señal original y los tiempos de exportación están acotados según el tipo de contenido.

En conjunto, el subsistema cierra el flujo de trabajo del sistema, ya que convierte los resultados producidos por los subsistemas Preprocesamiento y Análisis en artefactos persistentes y reutilizables fuera de la aplicación. Al integrarse con el núcleo a través del cierre del proyecto, el sistema asegura que el trabajo del investigador pueda conservarse antes de liberar los recursos de la sesión.

5.2. Vista Lógica

5.2.1. Propósito

La vista lógica describe la descomposición funcional de Gamma Lab 2.0 en clases, interfaces y abstracciones, junto con las responsabilidades de cada elemento y las relaciones estructurales que los vinculan. Esta vista describe los elementos que lo componen, la responsabilidad que asume cada uno de ellos y la forma en que colaboran para satisfacer los requerimientos funcionales.

5.2.2. Notación

Dado que Gamma Lab 2.0 evoluciona a partir de un sistema existente, cada elemento del modelo se clasifica según su estado de implementación. Los elementos representados en color verde se encontraban implementados en el código fuente de gammaLab 1.0, mientras que los elementos representados en color rojo están planificados para gammaLab 2.0 y deben construirse para satisfacer casos de uso o requerimientos pendientes. Esta distinción permite que el modelo sirva simultáneamente como descripción del sistema pre-existente y como especificación de la arquitectura objetivo, es decir gammaLab 2.0.

5.2.3. Visión general del modelo

La estructura lógica de Gamma Lab 2.0 responde al estilo arquitectónico de microkernel, en el cual un núcleo mínimo administra un conjunto de extensiones independientes que aportan la funcionalidad del sistema. Los elementos del modelo se organizan en seis agrupaciones. El núcleo administra el ciclo de vida de las extensiones y el registro de servicios. Los servicios del sistema proveen capacidades transversales de configuración, entrada y salida de archivos, exportación, persistencia de proyectos, estado compartido y ejecución de tareas. La capa de presentación agrupa los elementos de interfaz gráfica que coordinan la interacción con el usuario. El contrato de extensión define la interfaz que todo módulo funcional debe cumplir. Las extensiones implementan las operaciones y los análisis sobre las señales. Finalmente, el modelo de datos representa las señales, sus segmentaciones y los proyectos persistidos.

La Figura 3 presenta el diagrama de clases del sistema.

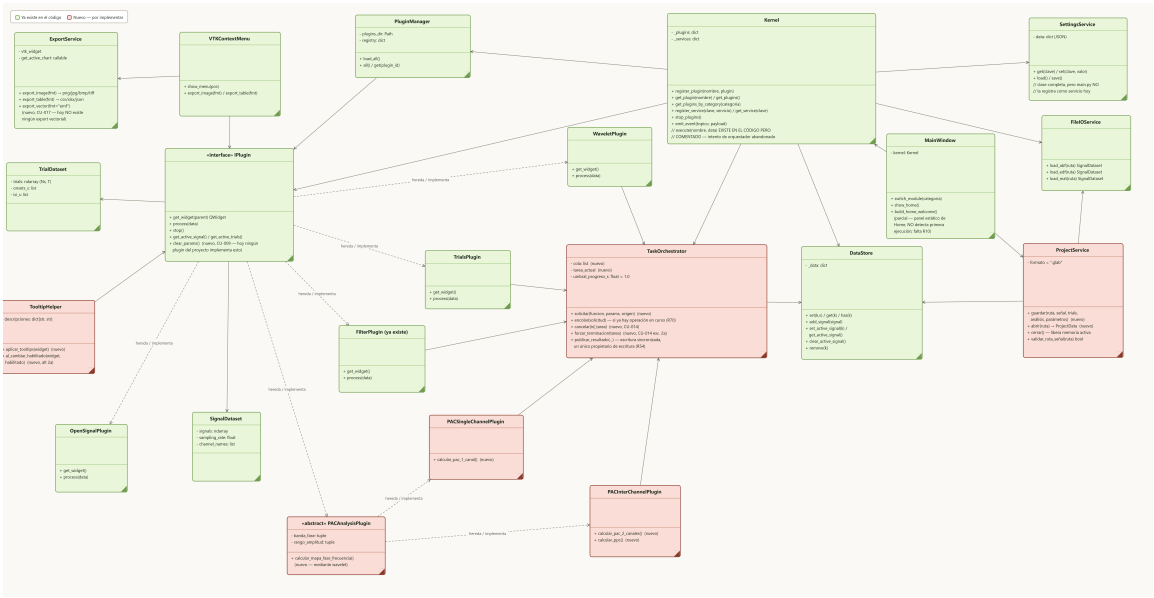


Figura 3: Diagrama de clases del sistema Gamma Lab 2.0. Los elementos en verde se encuentran implementados y los elementos en rojo están planificados.

5.2.4. Elementos del núcleo

Kernel El Kernel constituye el punto central de coordinación del sistema y se encuentra implementado. Mantiene dos registros internos: el registro de extensiones, indexadas por nombre, y el registro de servicios, indexados por clave. Sus operaciones permiten registrar una extensión (register_plugin), consultarla de forma individual o colectiva (get_plugin y get_plugins)

y obtener las extensiones pertenecientes a una categoría funcional (get_plugins_by_category), lo cual permite a la interfaz organizar los módulos por categorías. Asimismo, ofrece operaciones para registrar y localizar servicios por clave (register_service y get_service), para detener de forma ordenada todas las extensiones al finalizar la aplicación (stop_plugins) y para publicar eventos dirigidos a los componentes suscritos (emit_event). El Kernel se asocia con el PluginManager, el SettingsService, el FileIOService, el DataStore y el TaskOrchestrator, y administra las extensiones exclusivamente a través de la interfaz IPlugin.

PluginManager El PluginManager se encuentra implementado y es responsable de descubrir, cargar e instanciar las extensiones disponibles en el directorio de extensiones del sistema. Conserva la ubicación de dicho directorio y un registro de las extensiones descubiertas. Sus operaciones permiten cargar la totalidad de las extensiones (load_all) y consultarlas de forma colectiva o por identificador (all y get). El PluginManager depende de la interfaz IPlugin, de modo que únicamente admite elementos que cumplan dicho contrato.

5.2.5. Servicios del sistema

SettingsService El SettingsService se encuentra implementado y gestiona la configuración persistente de la aplicación. La configuración se mantiene en memoria como un diccionario y se almacena en disco en formato JSON. El servicio ofrece operaciones para consultar y modificar valores de configuración por clave (get y set) y para cargar y almacenar la configuración (load y save).

FileIOService El FileIOService se encuentra implementado y adapta los formatos de archivo soportados al modelo de datos interno del sistema. Sus operaciones load_abf, load_edf y load_mat leen archivos en los formatos ABF, EDF y MAT, respectivamente, y retornan en todos los casos un objeto SignalDataset. De esta manera, el resto del sistema permanece independiente de la representación de las señales en disco.

ExportService El ExportService se encuentra implementado y es responsable de exportar visualizaciones y resultados tabulares a formatos externos. Mantiene una referencia al componente de visualización y una función que provee la gráfica activa en cada momento. La operación export_image exporta la visualización a formatos de imagen ráster (PNG, JPG, BMP y TIFF) y la operación export_table exporta los resultados a formatos tabulares (CSV, XLSX y JSON). Se encuentra planificada la operación export_vector, que permitirá la exportación a formato vectorial EMF en respuesta al caso de uso CU-017.

DataStore El DataStore se encuentra implementado y mantiene en memoria el estado compartido de la sesión de trabajo, el cual comprende las señales cargadas, la señal activa, las mediciones y los resultados de los análisis. Constituye la fuente única de verdad del estado de la aplicación. Ofrece operaciones genéricas de almacenamiento, consulta y verificación por clave (set, get y has), operaciones para incorporar señales (add_signal), para establecer, consultar y liberar la señal activa (set_active_signal, get_active_signal y clear_active_signal) y para eliminar entradas del estado (remove).

TaskOrchestrator El TaskOrchestrator tiene la responsabilidad de coordinar la ejecución de las operaciones de cómputo solicitadas por las extensiones. Su propósito es garantizar que dichas operaciones se ejecuten de forma ordenada, que puedan ser canceladas por el usuario, que se

presente retroalimentación de progreso y que sus resultados se publiquen de manera consistente. Para ello mantendrá una cola de solicitudes pendientes, una referencia a la operación en ejecución y un umbral de tiempo, fijado en un segundo, a partir del cual se presentará el indicador de progreso.

La operación solicitar constituirá el punto de entrada único mediante el cual una extensión pide la ejecución de un cómputo. La operación encolar diferirá una solicitud cuando exista otra operación en curso, en cumplimiento del requerimiento R70. Las operaciones cancelar y forzar_terminacion permitirán, respectivamente, la cancelación de una tarea a petición del usuario y la terminación forzada de una tarea que no responda a la cancelación, conforme al caso de uso CU-014 y su excepción 2a. Por último, la operación publicar_resultado escribirá los resultados en el estado compartido de forma sincronizada. El TaskOrchestrator será el único componente autorizado para escribir en el DataStore, en cumplimiento del requerimiento R54.

ProjectService El ProjectService se encuentra implementado y gestiona la persistencia del espacio de trabajo en archivos de proyecto con extensión glab. Cada archivo de proyecto es un manifiesto en formato JSON que registra la señal de origen, las mediciones realizadas, la configuración de la segmentación en ensayos y el estado de los análisis. Durante el guardado, el archivo de señal se copia a la carpeta del proyecto, de modo que el proyecto resulta autocontenido y puede reabrirse en otro equipo. Durante la apertura, el servicio verifica la disponibilidad del archivo de señal referenciado, lo recarga mediante el FileIOService y restaura el estado de la sesión en el DataStore.

El servicio mantiene la carpeta del proyecto, la ruta del archivo de proyecto, el nombre del proyecto, un indicador de cambios pendientes de guardar y una función de notificación que informa a la interfaz sobre la apertura, el guardado, el cambio de nombre o el cierre del proyecto. Sus operaciones principales permiten guardar el proyecto en su ubicación actual (save), guardarlo en una nueva ubicación (save_as), abrir y restaurar un proyecto existente (open), y cerrar el proyecto activo liberando su estado (close). Adicionalmente, ofrece operaciones para asignar el nombre del proyecto (set_project_name), registrar la existencia de cambios pendientes (mark_dirty), verificar si existe un proyecto abierto (is_open) y generar un nombre predeterminado a partir del nombre de la señal (default_project_name). El ProjectService depende del FileIOService, del DataStore y del Kernel, y produce y consume objetos de tipo ProjectData.

5.2.6. Elementos de presentación

MainWindow La MainWindow se encuentra parcialmente implementada y constituye el contenedor principal de la interfaz gráfica. Es responsable de la navegación entre los módulos funcionales, de la presentación de la pantalla de inicio y de las operaciones de proyecto disponibles en el menú Archivo. Mantiene referencias al Kernel y al ProjectService. La operación switch_module presenta las extensiones correspondientes a la categoría seleccionada por el usuario, la operación (show_home) presenta la pantalla de inicio y la operación (build_home_welcome) construye el panel de bienvenida. Esta última se encuentra pendiente de incorporar la detección de la primera ejecución de la aplicación, requerida por el requerimiento R10.

VTKContextMenu El VTKContextMenu se encuentra implementado y provee el menú contextual asociado a las visualizaciones. Sus operaciones permiten presentar el menú en la posición indicada por el usuario (show_menu) y solicitar la exportación de la visualización como imagen o como tabla (export_image y export_table). Este elemento delega la exportación en el ExportService y opera sobre los componentes visuales provistos por las extensiones.

TooltipHelper El TooltipHelper se encuentra planificado y tendrá la responsabilidad de proveer ayuda contextual sobre los controles de la interfaz. Mantendrá un diccionario de descripciones asociadas a cada control. La operación (aplicar_tooltips) asignará dichas descripciones a los controles de un componente visual y la operación (al_cambiar_habilitado) actualizará la descripción de un control cuando cambie su disponibilidad, de modo que el usuario conozca el motivo por el cual un control se encuentra deshabilitado.

5.2.7. Contrato de extensión

IPlugin IPlugin es la interfaz que define el contrato que toda extensión debe cumplir para integrarse al sistema, y constituye el mecanismo principal de extensibilidad de la arquitectura. Se encuentra implementada y establece las siguientes operaciones: get_widget, que provee la interfaz gráfica de la extensión; process, que ejecuta el procesamiento propio de la extensión; stop, que detiene la extensión y libera sus recursos; y get_active_signal y get_active_trials, que proporcionan acceso a los datos de trabajo. IPlugin depende de las clases SignalDataset y TrialDataset.

Se encuentra planificada la incorporación al contrato de la operación clear_params, cuya finalidad es restablecer los parámetros de una extensión a sus valores predeterminados, conforme al caso de uso CU-009. Actualmente esta operación se encuentra implementada de forma individual en las extensiones FilterPlugin, TrialsPlugin y ArtifactRemovePlugin, pero aún no forma parte de la interfaz común.

5.2.8. Extensiones

Todas las extensiones del sistema realizan la interfaz IPlugin. La Tabla 17 resume las extensiones representadas en el modelo, su estado de implementación y su responsabilidad.

Cuadro 17: Extensiones representadas en la vista lógica.

Elemento	Responsabilidad
`OpenSignalPlugin`	Selección y carga de señales desde archivo.
`FilterPlugin`	Filtrado en frecuencia de la señal.
`WaveletPlugin`	Análisis en tiempo y frecuencia mediante la transformada wavelet.
`TrialsPlugin`	Segmentación de la señal en ensayos.
`PACAnalysisPlugin`	Comportamiento común del análisis de acoplamiento entre fase y amplitud (PAC).
`PACSingleChannelPlugin`	Cálculo del acoplamiento entre fase y amplitud dentro de un mismo canal.
`PACInterChannelPlugin`	Cálculo del acoplamiento entre fase y amplitud y de la consistencia de fase por pares (PPC) entre dos canales.

PACAnalysisPlugin es una clase abstracta que concentra los parámetros y el procedimiento comunes a todo análisis de acoplamiento entre fase y amplitud. Sus atributos definen la banda de frecuencia de la fase y el rango de frecuencia de la amplitud, y su operación calcular_mapa_fase_frecuencia obtiene el mapa de fase y frecuencia mediante la transformada wavelet. Las clases PACSingleChannelPlugin y PACInterChannelPlugin especializan este comporta-

miento. La primera incorpora la operación calcular_pac_1_canal, que calcula el acoplamiento dentro de un mismo canal, y la segunda incorpora las operaciones calcular_pac_2_canales y calcular_ppc, que calculan el acoplamiento y la consistencia de fase entre dos canales.

5.2.9. Modelo de datos

SignalDataset SignalDataset se encuentra implementada y representa una señal electrofisiológica multicanal. Sus atributos contienen la matriz de muestras de la señal, la frecuencia de muestreo expresada en hercios y los identificadores de los canales.

TrialDataset TrialDataset se encuentra implementada y representa una señal segmentada en ensayos. Sus atributos contienen la matriz de ensayos, cuyas dimensiones corresponden al número de muestras por ensayo y al número de ensayos, los instantes de inicio de cada ensayo expresados en segundos y los intervalos entre estímulos expresados en segundos.

ProjectData ProjectData es una estructura de datos implementada que representa en memoria el contenido de un archivo de proyecto. En la implementación actual se materializa como un diccionario serializado en formato JSON, producido por la operación save y retornado por la operación open del ProjectService. Sus atributos registran la versión del esquema del formato de proyecto, el nombre del proyecto, la fecha y hora del último guardado, la información de la señal de origen (nombre de archivo, formato, nombre y frecuencia de muestreo), las mediciones realizadas, la configuración de la segmentación en ensayos y los parámetros y el estado de las extensiones de análisis. La versión del esquema permite detectar incompatibilidades entre versiones del formato al momento de abrir un proyecto.

5.2.10. Relaciones estructurales

La Tabla 18 resume las relaciones entre los elementos del modelo lógico.

Cuadro 18: Relaciones estructurales de la vista lógica.

Origen	Tipo	Destino	Descripción
Kernel	Asociación	PluginManager	Delega la carga de extensiones.
Kernel	Asociación	SettingsService, FileIOService, DataStore, TaskOrchestrator	Registra y expone los servicios del sistema.
Kernel, PluginManager	Dependencia	IPlugin	Administran las extensiones a través de su contrato.
MainWindow	Asociación	Kernel, ProjectService	Accede a las extensiones, a los servicios y a las operaciones de proyecto.
ProjectService	Dependencia	FileIOService, DataStore	Recarga la señal y restaura el estado de la sesión.
ProjectService	Dependencia	ProjectData	Produce y consume el contenido del proyecto.
VTKContextMenu	Dependencia	ExportService	Delega la exportación.
VTKContextMenu, TooltipHelper	Dependencia	IPlugin	Operan sobre los componentes visuales de las extensiones.
IPlugin	Dependencia	SignalDataset, TrialDataset	Manipula el modelo de datos.
Extensiones concretas y PACAnalysisPlugin	Realización	IPlugin	Implementan el contrato de extensión.
PACSingleChannelPlugin, PACInterChannelPlugin	Generalización	PACAnalysisPlugin	Especializan el análisis de acoplamiento.
Extensiones de análisis	Dependencia	TaskOrchestrator	Solicitan la ejecución de operaciones de cómputo.
TaskOrchestrator	Asociación	DataStore	Publica los resultados en el estado compartido.

5.2.11. Justificación de las decisiones

La separación entre el núcleo y las extensiones mediante la interfaz IPlugin favorece la extensibilidad del sistema, ya que permite incorporar nuevos análisis, como los de acoplamiento entre fase y amplitud, sin modificar el código existente, en conformidad con el principio de abierto y cerrado.

La asignación exclusiva de la escritura sobre el estado compartido al TaskOrchestrator garantiza la consistencia del estado de la aplicación, al eliminar las condiciones de carrera entre operaciones concurrentes, conforme al requerimiento R54.

La centralización de la ejecución de los cómputos en el TaskOrchestrator mejora la capacidad de respuesta del sistema y el control que ejerce el usuario sobre él, puesto que habilita la ejecución ordenada de las operaciones (R70), su cancelación (CU-014) y la presentación de progreso, sin que cada extensión deba gestionar sus propios hilos de ejecución.

La concentración de la lectura de archivos en el FileIOService asegura la independencia del sistema respecto de los formatos de señal, y confina a un único elemento el impacto de incorporar nuevos formatos.

La persistencia de proyectos mediante el ProjectService y la estructura ProjectData produce proyectos autocontenidos y versionados. La inclusión de una copia de la señal de origen permite reabrir el trabajo en otro equipo, y la versión del esquema permite detectar incompatibilidades entre versiones del formato.

Por último, la jerarquía encabezada por PACAnalysisPlugin promueve la reutilización, al evitar la duplicación del cálculo del mapa de fase y frecuencia entre las variantes de análisis de uno y de dos canales.

5.2.12. Trazabilidad con los requerimientos

La Tabla 19 relaciona los elementos y operaciones del modelo lógico con los casos de uso y requerimientos que los originan.

Cuadro 19: Trazabilidad entre elementos de la vista lógica y requerimientos.

Elemento u operación	**Caso de uso o requerimiento**
IPlugin, operación clear_params	CU-009
TaskOrchestrator, operación cancelar	CU-014
TaskOrchestrator, operación forzar_terminacion	CU-014, excepción 2a
ExportService, operación export_vector	CU-017
ProjectService, operaciones save, save_as y close	CU-021, R07
ProjectService, operación open, y ProjectData	CU-023
TaskOrchestrator, operación publicar_resultado	R54
TaskOrchestrator, operación encolar	R70
MainWindow, operación build_home_welcome	R10
PACSingleChannelPlugin y PACInterChannelPlugin	CU-001 y CU-002

5.3. Vista de despliegue

5.3.1. Propósito

La vista de despliegue permite comprender cómo se organiza GammaLab 2.0 y cómo colaboran sus componentes para ofrecer las funcionalidades de procesamiento y análisis de señales neurofisiológicas. El Kernel y el PluginManager proporcionan la estructura necesaria para integrar las extensiones, mientras que los servicios de soporte se encargan de gestionar los datos, los archivos, la configuración, los proyectos y la exportación de resultados.

Por su parte, los plugins reúnen las herramientas de entrada y salida, preprocesamiento, análisis y medición. A estos componentes se suman nuevas incorporaciones, como el TaskOrchestrator, orientado a coordinar la ejecución de tareas; el ProjectService, destinado a conservar y recuperar el estado del trabajo; y el TooltipHelper, que busca facilitar la comprensión de las herramientas disponibles.

En conjunto, esta organización permite que GammaLab 2.0 evolucione a partir de su versión anterior, reutilice funcionalidades existentes e incorpore nuevas capacidades de manera ordenada. La separación de responsabilidades contribuye a que el sistema sea más fácil de mantener y ampliar, al tiempo que establece una estructura que puede acompañar el crecimiento de las necesidades de análisis del laboratorio.

5.3.2. Notación

La leyenda incluida en la Figura 4 permite distinguir los componentes según su estado y su relación con el alcance definido para GammaLab 2.0. Los colores se interpretan de la siguiente manera:

- **Azul - Núcleo existente:** representa los componentes centrales que ya forman parte de la estructura de la aplicación, como `MainWindow` y `Kernel`.
- **Amarillo - Servicio o plugin existente:** identifica los servicios y plugins que se reutilizan de la versión anterior, como `DataStore`, `FileIOService`, `SettingsService` y las extensiones funcionales existentes.
- **Rojo - Nuevo:** identifica los componentes incorporados o previstos para GammaLab 2.0, entre ellos `ProjectService`, `TaskOrchestrator` y `TooltipHelper`.
- **Gris - Fuera del alcance de los 23 casos de uso:** representa los componentes que se muestran como referencia funcional o documental, pero que no forman parte del alcance específico considerado en dichos casos de uso.

Esta distinción permite identificar con mayor facilidad qué partes de la arquitectura se apoyan en funcionalidades existentes y cuáles corresponden a nuevas incorporaciones. También ayuda a comprender que el diagrama representa tanto la estructura reutilizada como la organización propuesta para ampliar el sistema.

5.3.3. Visión general del modelo

GammaLab 2.0 sigue una arquitectura basada en el estilo *Microkernel*. En esta estructura, el Kernel funciona como el punto central de coordinación, encargado de mantener el registro de los plugins y servicios disponibles. A su alrededor se encuentran los componentes que permiten cargar señales, realizar operaciones de preprocesamiento, ejecutar análisis, obtener medidas y exportar resultados.

La organización modular permite que cada componente se concentre en una responsabilidad específica, sin que toda la lógica de la aplicación dependa de un único módulo. Esto facilita el mantenimiento del sistema y permite ampliar sus capacidades mediante la incorporación de nuevas funcionalidades. Además, el diagrama distingue entre los componentes existentes, aquellos que se plantean como nuevas incorporaciones para GammaLab 2.0 y los elementos que se encuentran fuera del alcance de los 23 casos de uso considerados.

A continuación, se describe la función de cada grupo de componentes y su participación en el funcionamiento general del sistema.

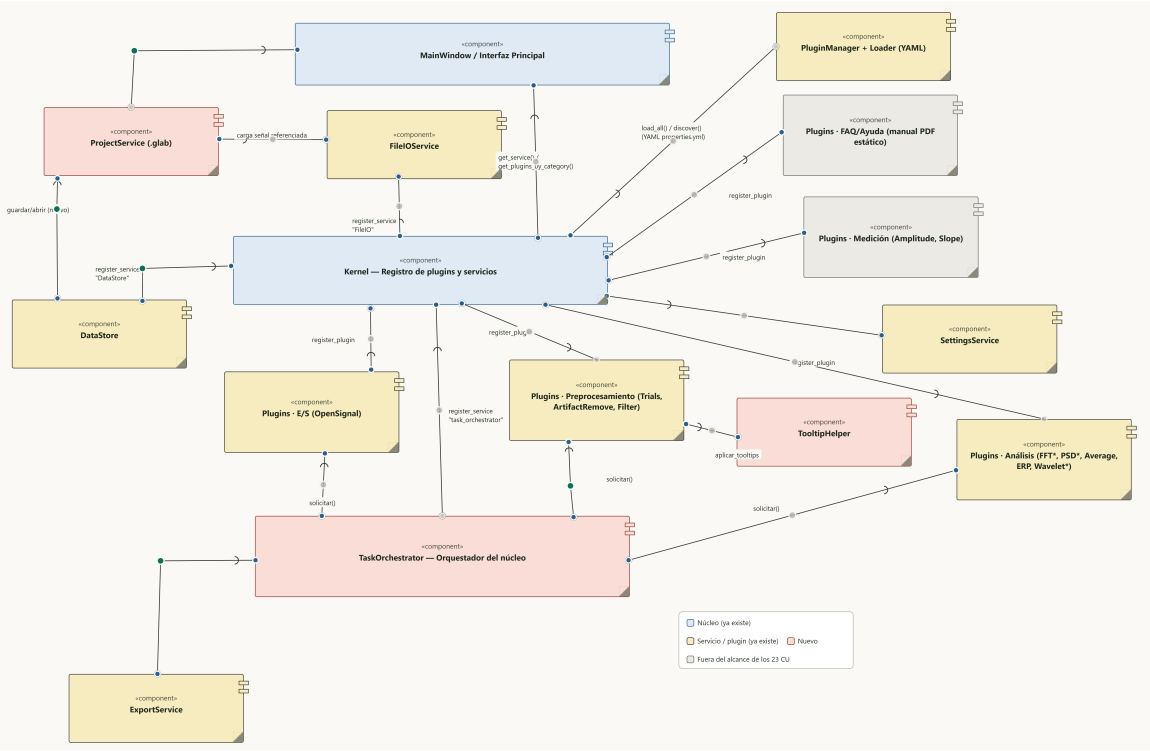


Figura 4: Vista de desarrollo de GammaLab 2.0: componentes organizados por paquetes funcionales (`io`, `preprocessing`, `analysis`, `measure` y `faq`), junto con los componentes transversales de la arquitectura.

5.3.4. Componentes principales

El Kernel es el núcleo de GammaLab 2.0 y permite que los diferentes componentes trabajen de manera coordinada. Su función principal es mantener el registro de los plugins y servicios disponibles, de modo que puedan ser localizados y utilizados por los módulos que los necesiten. Gracias a esta organización, los componentes pueden acceder a las funcionalidades compartidas sin depender directamente de los detalles internos de su implementación.

Entre sus principales responsabilidades se encuentran:

- Registrar los plugins y servicios mediante operaciones como `register_plugin` y `register_service`.
- Consultar los plugins disponibles, tanto de manera individual como por categorías funcionales.
- Facilitar el acceso a los servicios registrados mediante la operación `get_service`.
- Coordinar la integración de los diferentes módulos de la aplicación.
- Gestionar el ciclo de vida de los plugins y facilitar la comunicación entre los componentes.

En el diagrama, el Kernel se encuentra conectado con la interfaz principal, el gestor de plugins, los servicios de soporte y las distintas extensiones funcionales. Esta posición refleja su papel dentro de la arquitectura: servir como punto de acceso y coordinación, manteniendo una separación entre las funcionalidades centrales y los módulos que amplían las capacidades del sistema.

La interacción con el usuario se realiza mediante el componente `MainWindow`, que corresponde a la interfaz gráfica principal. Desde ella, el usuario puede acceder a las herramientas disponibles,

seleccionar las operaciones que desea realizar y consultar los resultados obtenidos. Para ofrecer estas funcionalidades, la interfaz se comunica con el Kernel, que permite localizar los plugins y servicios correspondientes. Así, la interfaz se concentra en la interacción con el usuario, mientras que el procesamiento se delega en los componentes especializados.

Por otra parte, el `PluginManager` se encarga de descubrir, cargar e instanciar los plugins disponibles. Este componente permite incorporar las extensiones al sistema y ponerlas a disposición de la aplicación. Para ello, utiliza operaciones como `load_all()` y `discover()`, junto con los mecanismos de configuración y metadatos asociados a los plugins.

En conjunto, el Kernel, la MainWindow y el PluginManager conforman la estructura principal sobre la que se integran las demás funcionalidades de GammaLab 2.0.

5.3.5. Servicios de soporte y gestión de datos

Además de los componentes centrales, GammaLab 2.0 cuenta con servicios que resuelven necesidades comunes de la aplicación. Estos servicios permiten compartir información, administrar la configuración y gestionar archivos y resultados sin que cada plugin tenga que implementar estas funciones por separado.

El `DataStore` mantiene el estado compartido de la aplicación y proporciona acceso a los datos que utilizan los diferentes módulos durante el procesamiento y el análisis. Su función es especialmente importante porque permite que las operaciones se realicen sobre la información disponible en la sesión de trabajo, evitando que cada componente gestione de manera independiente sus propios datos.

El `FileIOService` se encarga de leer y validar los archivos que contienen las señales neurofisiológicas. De esta manera, centraliza el acceso a los datos de entrada y permite que los componentes que requieren una señal puedan utilizar un mecanismo común para cargarla.

Por su parte, el `SettingsService` administra la configuración general de la aplicación y las preferencias del usuario. Este servicio permite consultar y modificar los parámetros de configuración, manteniéndolos disponibles para los componentes que los necesiten.

El `ExportService` se ocupa de gestionar la exportación de los resultados obtenidos durante el análisis, facilitando su almacenamiento y posterior utilización. De esta forma, la generación de archivos de salida se mantiene separada de los algoritmos que realizan los cálculos.

Finalmente, el `ProjectService` es uno de los nuevos componentes contemplados para GammaLab 2.0. Su propósito es permitir que el usuario guarde y recupere su espacio de trabajo mediante archivos con extensión `.glab`. Estos proyectos conservan información como la señal de referencia, los *trials*, los análisis realizados y los parámetros asociados.

Para cumplir esta función, el ProjectService se relaciona con el DataStore y el FileIOService. El primero permite acceder a la información de la sesión, mientras que el segundo facilita la recarga de la señal original cuando se abre un proyecto que la necesita. De esta manera, el usuario puede continuar su trabajo sin tener que reconstruir manualmente todo el estado de la sesión.

En conjunto, estos servicios proporcionan las capacidades necesarias para gestionar los datos y la configuración, trabajar con archivos y conservar los resultados de las operaciones realizadas en GammaLab 2.0.

5.3.6. Orquestador del núcleo

El `TaskOrchestrator` es un componente nuevo cuyo propósito es coordinar la ejecución de las tareas de procesamiento y análisis. Su incorporación responde a la necesidad de gestionar operaciones que pueden requerir un tiempo considerable de cálculo sin afectar innecesariamente la interacción del usuario con la interfaz gráfica.

En lugar de que cada plugin tenga que encargarse directamente de toda la coordinación de su ejecución, puede solicitar la operación correspondiente al orquestador. Este componente se encarga de coordinar la tarea y su interacción con los servicios necesarios, manteniendo separadas las responsabilidades de procesamiento y gestión de la ejecución.

Entre sus responsabilidades previstas se encuentran:

- Recibir las solicitudes de ejecución realizadas por los plugins.
- Coordinar tareas de procesamiento, análisis y exportación.
- Facilitar el seguimiento de las tareas y su cancelación cuando corresponda.
- Contribuir a que las operaciones de larga duración no bloqueen la interfaz gráfica.
- Coordinar el acceso a los servicios necesarios para completar cada operación.

Como se muestra en la Figura 4, el TaskOrchestrator se relaciona con los plugins de entrada y salida, preprocesamiento y análisis. Estos componentes pueden solicitar la ejecución de las operaciones correspondientes, mientras que el orquestador se encarga de coordinar su procesamiento. También se relaciona con el ExportService para gestionar las tareas asociadas a la generación de resultados.

Su incorporación busca mejorar la capacidad de respuesta de la aplicación y facilitar la ejecución de operaciones computacionalmente exigentes. No obstante, las capacidades efectivamente disponibles dependerán de la implementación de los mecanismos de ejecución previstos para este componente.

5.3.7. Plugins funcionales

Los plugins son los componentes que proporcionan las principales herramientas de procesamiento y análisis de GammaLab 2.0. Cada uno se encarga de una funcionalidad específica y se integra con el resto del sistema mediante el mecanismo de registro administrado por el Kernel.

Esta organización permite que las funcionalidades se agrupen según el propósito de las operaciones que realizan. De acuerdo con el diagrama, los plugins se distribuyen en las siguientes categorías:

- **Plugins de entrada y salida (`io`):** permiten cargar señales desde archivos compatibles. En esta categoría se encuentra OpenSignal, encargado de proporcionar las funcionalidades de entrada necesarias para incorporar los datos a la aplicación. Estos plugins utilizan los servicios correspondientes para acceder a las señales y ponerlas a disposición de los demás módulos.
- **Plugins de preprocesamiento (`preprocessing`):** incluyen `Trials`, `ArtifactRemove` y `Filter`. Estas herramientas permiten preparar las señales antes de analizarlas, mediante operaciones como la segmentación en ensayos o *trials*, la eliminación o corrección de artefactos y la aplicación de filtros.

- **Plugins de medición (`measure`):** incluyen `Amplitude` y `Slope`. Su función es obtener medidas cuantitativas a partir de las señales o de los resultados disponibles. La amplitud permite caracterizar la magnitud de una respuesta, mientras que la pendiente describe su variación en un intervalo determinado.
- **Plugins de análisis (`analysis`):** incluyen `FFT`, `PSD`, `Average`, `ERP` y `Wavelet`. Estas herramientas permiten estudiar las señales desde diferentes perspectivas. Los análisis temporales incluyen el promediado y los potenciales relacionados con eventos; los análisis en frecuencia incluyen la transformada rápida de Fourier y la densidad espectral de potencia; y los análisis tiempo-frecuencia permiten estudiar cómo varían las características espectrales a lo largo del tiempo.

Estas categorías se complementan entre sí dentro del flujo de trabajo de la aplicación. Por ejemplo, una señal puede cargarse mediante un plugin de entrada, prepararse con las herramientas de preprocesamiento y utilizarse posteriormente en un análisis o una medición. Los resultados pueden consultarse desde la interfaz y, cuando corresponda, exportarse para su utilización posterior.

La separación de las funcionalidades en plugins permite que cada módulo tenga una responsabilidad delimitada y que el sistema pueda ampliarse con nuevas herramientas sin necesidad de modificar toda su estructura. Para ello, las extensiones deben respetar los mecanismos de integración definidos por la arquitectura.

5.3.8. Documentación de ayuda

El componente `TooltipHelper` se incorpora con el propósito de facilitar el uso de las herramientas disponibles en GammaLab 2.0. Su función consiste en proporcionar descripciones breves sobre los plugins, de manera que el usuario pueda comprender qué hace cada herramienta antes de utilizarla.

En el diagrama, este componente se relaciona con los plugins de preprocesamiento mediante la operación `aplicar_tooltips`. Esta relación permite asociar información descriptiva a las herramientas y presentarla mediante ayudas emergentes en la interfaz.

Adicionalmente, el diagrama incluye un componente de FAQ y ayuda asociado a un manual PDF estático. Este recurso permite consultar documentación complementaria cuando el usuario necesita información más detallada sobre el funcionamiento de la aplicación.

Ambos componentes cumplen funciones diferentes, pero complementarias. Mientras que el TooltipHelper ofrece orientación inmediata durante la interacción con las herramientas, la documentación de ayuda proporciona explicaciones más amplias que pueden consultarse según las necesidades del usuario.

De esta manera, la arquitectura contempla tanto la asistencia contextual dentro de la interfaz como el acceso a documentación de referencia.

5.3.9. Flujo general

La interacción entre los componentes permite desarrollar las operaciones de GammaLab 2.0 desde la selección de una herramienta hasta la obtención y conservación de los resultados. Aunque el recorrido puede variar según la funcionalidad utilizada, es posible identificar las siguientes etapas generales:

1. **Interacción con la interfaz:** el usuario inicia una operación desde la `MainWindow`, seleccionando la funcionalidad que necesita.
2. **Identificación del componente:** el Kernel y el PluginManager permiten localizar los plugins y servicios registrados que corresponden a la operación solicitada.
3. **Coordinación de la tarea:** cuando la operación requiere procesamiento, el TaskOrchestrator coordina su ejecución y el acceso a los servicios necesarios.
4. **Procesamiento de la señal:** el plugin correspondiente realiza la operación solicitada, que puede consistir en cargar, preprocesar, analizar o medir una señal.
5. **Gestión de resultados:** los resultados obtenidos pueden actualizar el estado compartido de la aplicación, presentarse en la interfaz o exportarse mediante los servicios correspondientes.
6. **Persistencia del trabajo:** cuando el usuario guarda un proyecto, el ProjectService conserva el estado del espacio de trabajo para permitir su recuperación posterior.

Este recorrido describe la colaboración general entre los componentes, pero no representa una secuencia obligatoria para todas las operaciones. Por ejemplo, al abrir un proyecto intervienen principalmente el ProjectService y el FileIOService para recuperar la información almacenada y, cuando es necesario, cargar nuevamente la señal de referencia. En cambio, la exportación de resultados requiere la coordinación de la tarea correspondiente y la intervención del ExportService.

En términos generales, cada componente participa en la operación de acuerdo con su responsabilidad. La interfaz recibe las acciones del usuario, el Kernel permite localizar las funcionalidades disponibles, los plugins ejecutan las operaciones específicas y los servicios proporcionan el acceso a los datos y recursos necesarios. Cuando la operación lo requiere, el orquestador coordina su ejecución y los resultados se presentan, almacenan o exportan según corresponda.

Esta distribución de responsabilidades permite que los componentes colaboren sin concentrar toda la lógica del sistema en un único módulo y facilita la incorporación de nuevas funcionalidades.

Dos hallazgos relevantes para la planificación de la implementación:

- Los paquetes `measure/` (Amplitude, Slope) y `faq/` (manual PDF estático) existen en el código pero **ninguno** de los 23 casos de uso los cubre — se muestran en gris neutro para delimitar la frontera del alcance actual.
- `TooltipHelper` depende de los plugins de preprocesamiento y análisis por igual (CU-010): su ausencia hoy no es aislada, sino transversal a ∼15 plugins que carecen de *tooltips*, salvo dos excepciones puntuales ya presentes en `TrialsPlugin` y `RelativePSDPlugin`.

5.4. Vista de Procesos

5.4.1. Propósito

La vista de procesos permite comprender cómo se organiza el flujo de trabajo entre el investigador y el sistema de GammaLab v2.0 durante una sesión de trabajo, desde el inicio de la aplicación hasta la exportación de los resultados, guardado y el cierre del proyecto.

Asimismo, el diagrama que se presentara más adelante permite identificar las responsabilidades del usuario y del sistejma, las descisiones que condicionan la ejecución del flujo de trabajo y los mecanismos mediante los cuales se gestionan los cálculos asociados al procesamiento de señales.

Esta vista permite el entendimiento del comportamiento dinámico del sistema desde una perspectiva arquitectónica, mostrando cómo el flujo que siga el investigador desencadena las operacione sinternas y los resultados generados se ponene a disposición de este.

5.4.2. Notación

La leyenda del diagrama distingue los elementos según su estado de implementación:

- **Verde - Ya existe:** representa las actividades o funcionalidades que ya se encuentran disponibles en la aplicación.
- **Rojo - Nuevo:** identifica las actividades asociadas a componentes o funcionalidades nuevas contempladas para GammaLab 2.0.
- **Gris - Mixto:** corresponde a actividades cuyo funcionamiento combina elementos existentes con nuevas incorporaciones, según la descripción asociada al diagrama.

5.4.3. Visión general del modelo

El modelo representa el flujo de trabajo general de GammaLab 2.0, desde el inicio de la aplicación hasta el cierre de la sesión. Su propósito es mostrar cómo se relacionan las acciones del investigador con los procesos internos del sistema y cómo se desarrolla una sesión de análisis de señales neurofisiológicas.

El flujo comienza con la inicialización de la aplicación, durante la cual se registran los servicios disponibles y se descubren los plugins mediante los mecanismos de configuración YAML. Posteriormente, el investigador puede elegir entre abrir un proyecto previamente guardado o comenzar una sesión cargando una señal desde un archivo. En ambos casos, el sistema prepara los datos y los publica en el `DataStore` para que puedan utilizarse en las etapas posteriores.

Una vez disponible la señal, el investigador puede realizar operaciones de preprocesamiento, ejecutar los análisis necesarios e interactuar visualmente con los resultados. El modelo también contempla decisiones que permiten regresar a etapas anteriores para continuar trabajando con la señal activa o avanzar hacia la exportación de resultados y el guardado del proyecto.

Para representar estas interacciones, el diagrama se organiza en dos carriles: *Investigador* y *Sistema*. El primero agrupa las acciones y decisiones que corresponden al usuario, mientras que el segundo representa las operaciones que ejecuta la aplicación para responder a dichas acciones. Esta separación permite identificar las responsabilidades de cada participante y comprender cómo se coordinan durante una sesión de trabajo.

La Figura 5 presenta el modelo completo, incluyendo las decisiones que controlan el flujo, las actividades principales y las relaciones entre las acciones del investigador y los procesos internos de GammaLab 2.0.

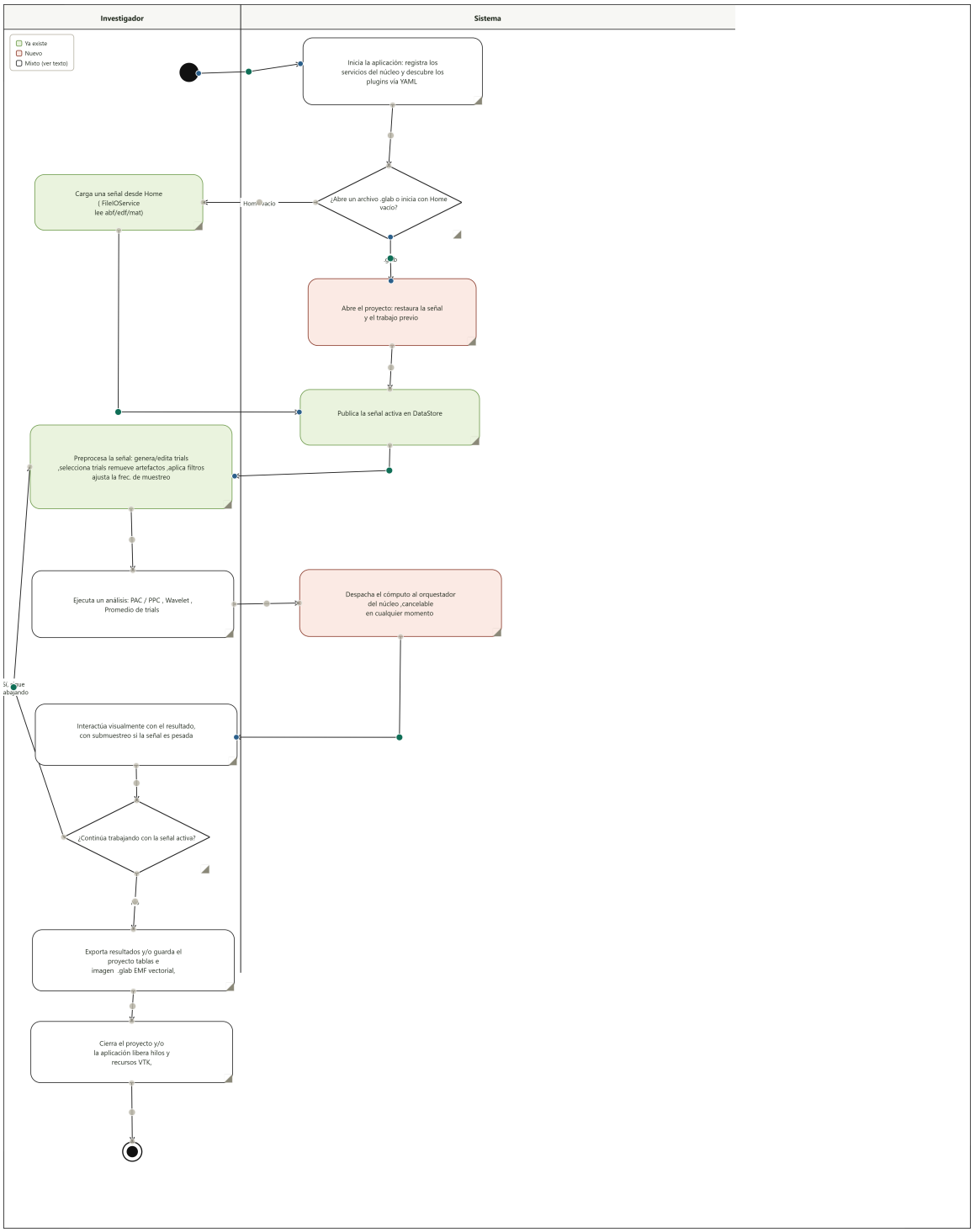


Figura 5: Flujo maestro de sesión de Gamma Lab 2.0, con carriles Investigador/Sistema. Verde = ya existe, rojo = nuevo, blanco = mixto.

5.4.4. Inicio de la aplicación

El flujo de ejecución comienza con la inicialización de GammaLab v2.0. Durante esat aetapa, el sistema premara los componentes centrales necesarios para el funcinamiento del sistema y registra los servicios que estará disponibles para lso demás módulos. Luego, se lleva a cabo la descubrimiento de los plugins mediante los mecanismos de configuración YAML. Esto permite identificar las extensione disponibles y peraparaa su integración cpon el núcleo del sistema. La

inicialización establece las condiciones necesarias para que el investigador pueda acceder a las funcionalidades de carga, procesamiento y análisis de señales. Asimismo, prepara la infraestructura de servicios que permite compartir datos y coordinar las operaciones ejecutadas durante la sesión.

5.4.5. Apertura de proyecto o carga de una señal

5.4.6. Procesamiento de señales

5.4.7. Ejecución de análisis y coordinación de tareas

5.4.8. Visualización e interacción con los resultados

5.4.9. Continuación del trabajo, guardado y cierre

5.5. Vista física

5.5.1. Propósito

La vista física describe el entorno de ejecución de Gamma Lab 2.0 y la forma en que el sistema se relaciona con él. Representa la infraestructura sobre la que se despliegan los componentes de software y los artefactos que el sistema intercambia con su entorno, de modo que hace explícitos los límites de persistencia del sistema y los datos que entran y salen de él.

5.5.2. Notación

El diagrama sigue la notación de despliegue definida en la especificación UML 2.5.1 del Object Management Group [17]. El equipo físico donde se ejecuta el sistema se representa como un nodo con el estereotipo «device», y dentro de él se ubican los componentes de software desplegados. Los archivos que el sistema consume o produce se representan como elementos con el estereotipo «artifact». Las relaciones entre el nodo y los artefactos son dependencias, representadas con líneas punteadas, y su etiqueta indica el tipo de acceso (lectura, escritura o ambos) y la operación del sistema que lo origina.

Al igual que en la vista lógica, cada elemento del diagrama se clasifica según su origen. Los elementos sin color de fondo provienen de Gamma Lab 1.0, mientras que los elementos en color rojo se incorporaron en Gamma Lab 2.0.

5.5.3. Visión general del modelo

El modelo físico de Gamma Lab 2.0 está formado por un único nodo, la estación de trabajo del investigador, en el que se despliegan todos los componentes del sistema. Esto corresponde a una aplicación de escritorio monousuario que se ejecuta como un solo proceso local, sin servidores, sin comunicación en red y sin dependencias de servicios externos. Dentro del nodo, la interfaz principal concentra la interacción con el usuario. El núcleo, junto con el gestor de extensiones, administra el registro de servicios y el ciclo de vida de los módulos. El almacén de datos mantiene en memoria el estado compartido de la sesión. El orquestador de tareas ejecuta las operaciones de cómputo pesado en hilos trabajadores dentro del mismo proceso. El servicio de proyectos gestiona la persistencia y la restauración del trabajo del investigador. Por último, las extensiones implementan las funcionalidades de preprocesamiento, análisis y visualización.

Este modelo de despliegue tiene dos implicaciones arquitectónicas. La primera es que el desempeño del sistema depende por completo de los recursos de la estación de trabajo, en particular de la memoria disponible y de la capacidad de procesamiento. Por eso la liberación explícita de recursos y la ejecución concurrente controlada por el núcleo pasan a ser responsabilidades centrales de la arquitectura. La segunda es que el sistema de archivos es la única frontera de integración del sistema. Por esa frontera, el sistema se relaciona con cinco artefactos. Los archivos de señal son la fuente de datos de cada sesión y se leen durante la carga. El archivo de proyecto almacena y restaura el estado del trabajo mediante una referencia a la señal de origen, sin duplicar sus datos. Los archivos exportados contienen los resultados numéricos y gráficos de los análisis. El archivo de configuración conserva las preferencias de la aplicación. Los descriptores de cada extensión se leen al iniciar la aplicación para descubrir y registrar los módulos disponibles, lo que permite incorporar nuevas extensiones sin modificar el núcleo.

La Figura 6 presenta el diagrama de despliegue del sistema.

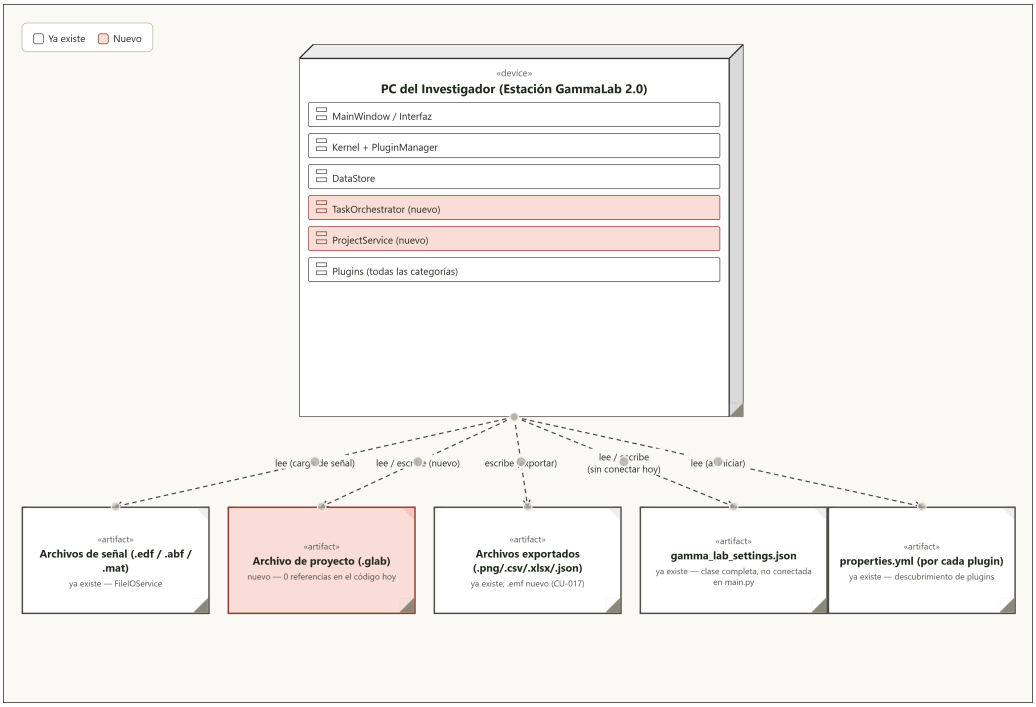


Figura 6: Vista física de Gamma Lab 2.0: estación única de escritorio y su superficie de E/S.

5.5.4. Componentes internos

Los componentes internos del nodo se instancian durante el arranque de la aplicación y permanecen activos hasta su cierre. Cada uno ocupa una posición distinta dentro del proceso: algunos se ejecutan en el hilo principal, otros en hilos trabajadores, y otros actúan como estructuras compartidas entre ambos.

- **Núcleo y gestor de extensiones (Kernel + PluginManager):** el núcleo es el componente mínimo del sistema. No implementa funcionalidad de dominio, sino que provee los servicios que las extensiones necesitan para operar. Es el primer componente que se instancia. Crea y registra el resto de los servicios, y a través del gestor de extensiones carga cada módulo disponible. Durante la ejecución actúa como punto de resolución de dependencias: las extensiones le solicitan los servicios que requieren, en lugar de construirlos.
- **Interfaz principal (MainWindow):** se construye después de que el núcleo termina el

registro y se ejecuta en el hilo principal. Recibe los eventos del usuario, activa y detiene las extensiones según la navegación y es el único componente que actualiza la presentación gráfica.

- **Almacén de datos (DataStore):** se instancia como un servicio único del núcleo y lo comparten todas las extensiones. Es el punto de encuentro entre el hilo principal, que lee el estado para presentarlo, y los hilos trabajadores, que escriben los resultados. Por eso su acceso se sincroniza y cada dato tiene un único propietario de escritura.
- **Orquestador de tareas (TaskOrchestrator):** se registra en el núcleo durante el arranque y mantiene las colas de ejecución durante toda la sesión. Por cada solicitud crea un hilo trabajador, lo supervisa durante el cómputo y lo libera al terminar, ya sea por finalización, error o cancelación.
- **Servicio de proyectos (ProjectService):** se registra en el núcleo como servicio compartido y solo se invoca en los momentos del ciclo de vida del proyecto: apertura, guardado y cierre.
- **Extensiones (Plugins):** cada extensión se carga en el arranque, pero su interfaz se construye la primera vez que el usuario la activa y se reutiliza en las activaciones siguientes. Al desactivarse, libera sus recursos gráficos y sus detectores de eventos. La instancia permanece registrada en el núcleo.

En ejecución, la interacción entre estos componentes sigue un ciclo constante. La interfaz recibe una acción del usuario y la dirige a la extensión activa. La extensión obtiene del núcleo el orquestador y le entrega la operación como una tarea. El orquestador la ejecuta en un hilo trabajador, que publica el resultado en el almacén de datos mediante escritura sincronizada. Por último, la interfaz recibe la notificación y actualiza la presentación desde el hilo principal. Gracias a este ciclo, ningún cómputo pesado ocupa el hilo principal y ninguna extensión escribe directamente sobre la presentación. Al cerrar la aplicación, el recorrido se invierte: el orquestador cancela o espera las tareas en curso, las extensiones se detienen y liberan sus recursos, y el núcleo finaliza el proceso.

5.5.5. Artefactos externos

Cada artefacto externo tiene un componente interno responsable de su acceso y un momento definido de la ejecución en el que se lee o se escribe. Las operaciones de lectura y escritura de mayor volumen se ejecutan a través del orquestador, fuera del hilo principal.

- **Archivos de señal (.edf, .abf, .mat):** se leen al cargar una señal o al restaurar un proyecto. Antes de cerrar el proyecto activo, el sistema valida el formato del archivo, de modo que un archivo no soportado o corrupto no altera la sesión en curso. La lectura y la construcción del conjunto de datos se ejecutan en un hilo trabajador, y el resultado se registra después en el almacén de datos.
- **Archivo de proyecto (.glab):** lo lee y escribe exclusivamente el servicio de proyectos. Se escribe al guardar o al cerrar un proyecto, y se lee al abrirlo. En la apertura, el servicio verifica que el archivo de señal referenciado siga en la ruta registrada antes de iniciar la carga. Si el archivo no está, el usuario puede localizarlo manualmente.
- **Archivos exportados (.png, .csv, .xlsx, .json, .emf):** se escriben a solicitud del usuario a partir del resultado activo en el almacén de datos. Si la operación supera un segundo, la interfaz muestra un indicador de progreso. Si la escritura falla por permisos o por falta de espacio, se informa la causa y el estado de la sesión se mantiene sin cambios.

- **Archivo de configuración (gamma_lab_settings.json):** contiene las preferencias de la aplicación y se lee y escribe a través del servicio de configuración del núcleo, que entrega una única instancia de esas preferencias a todos los módulos.
- **Descriptores de extensión (properties.yml):** el gestor de extensiones los lee una sola vez durante el arranque, antes de construir la interfaz. Cada descriptor determina si su extensión se carga y en qué categoría de la interfaz se ubica.

6. Atributos de calidad

6.1. Escenarios de calidad

6.1.1. Escenario de Calidad 1: Integración de un Plugin Externo

Cuadro 20: Escenario de calidad 1 — Integración de un plugin externo

Aspecto	**Descripción**
Atributo de calidad	Soporte
Requisito	El sistema debe permitir integrar plugins externos sin recompilar el núcleo y sin afectar módulos existentes.
Fuente del estímulo	Desarrollador externo
Estímulo	Se agrega un nuevo plugin a la carpeta de Plugins siguiendo el formato definido por el Kernel.
Artefacto	Módulo core del sistema (Kernel) y módulo de plugins.
Ambiente	Sistema en operación normal con múltiples plugins ya registrados.
Respuesta	El sistema detecta, valida y registra el plugin, exponiéndolo en la interfaz de usuario.
Medida de Respuesta	Plugin visible en la UI en menos de 1 minuto. Ningún plugin existente debe fallar tras la carga.
Trazabilidad	NFR-S-05 (R77–R78)

6.1.2. Escenario de Calidad 2: Carga de Señal de Gran Tamaño

Cuadro 21: Escenario de calidad 2 — Carga de señal de gran tamaño

Aspecto	**Descripción**
Atributo de calidad	Rendimiento
Requisito	El sistema debe gestionar señales de gran tamaño sin bloqueos ni degradación significativa del rendimiento.
Fuente del estímulo	Usuario final
Estímulo	Se carga un archivo ABF/EDF de gran tamaño (ej. 1–2 GB, múltiples canales).
Artefacto	FileIOService, DatasetService (DataStore), servicio de tareas del núcleo, interfaz de usuario.
Ambiente	Computador con carga moderada y otros procesos del sistema activos.
Respuesta	La señal se carga correctamente, sin duplicación innecesaria de datos y sin bloqueos de la interfaz; la lectura corre en el servicio de tareas, fuera del hilo de la UI.
Medida de Respuesta	Visualización inicial en menos de 8 segundos para este caso de estrés (el umbral base de carga es ≤ 2 s para el archivo típico de referencia). Uso de CPU menor a 80 %. Sin cierres inesperados ni latencias mayores a 5 segundos.
Trazabilidad	NFR-P-03 (R48), NFR-P-14 (R72), NFR-P-10 (R56)

6.1.3. Escenario de Calidad 3: Manejo de Errores en Tiempo de Ejecución

Cuadro 22: Escenario de calidad 3 — Manejo de errores en tiempo de ejecución

Aspecto	**Descripción**
Atributo de calidad	Confiabilidad
Requisito	El sistema debe manejar excepciones en los plugins sin comprometer la estabilidad del sistema.
Fuente del estímulo	Sistema o entorno
Estímulo	Un plugin genera una excepción interna durante un análisis (FFT, PSD, Wavelet, ERP, PAC, etc.).
Artefacto	Módulo de plugins, Kernel, servicios asociados a cada plugin.
Ambiente	Análisis en curso con señales cargadas y otros plugins activos.
Respuesta	El sistema captura la excepción, informa al usuario y evita un cierre inesperado de la aplicación, manteniendo el Kernel operando de manera adecuada.
Medida de Respuesta	0 % de pérdida de datos. Continuidad del sistema garantizada. Mensaje de error mostrado en menos de 2 segundos.
Trazabilidad	R47

6.1.4. Escenario de Calidad 4: Exportación de Datos o Gráficas

Cuadro 23: Escenario de calidad 4 — Exportación de datos o gráficas

Aspecto	**Descripción**
Atributo de calidad	Confiabilidad
Requisito	El sistema debe exportar gráficos y datos sin errores, incluso si se presentan fallas externas por disco, permisos, memoria u otros motivos.
Fuente del estímulo	Usuario final
Estímulo	El usuario exporta una tabla, medición o gráfico en PNG/CSV/JSON.
Artefacto	ExportService (registrado en el núcleo por inversión de control), interfaz de usuario, sistema de archivos.
Ambiente	Sistema operando con múltiples análisis realizados previamente sin guardar.
Respuesta	El archivo se genera correctamente o, en caso de error, se notifica al usuario sin afectar el estado del sistema.
Medida de Respuesta	Tiempo de exportación menor a 3 segundos. 0 % de pérdida de datos. Notificación clara ante error.
Trazabilidad	NFR-P-07 (R51), NFR-P-12 (R57), NFR-S-03 (R70)

6.1.5. Escenario de Calidad 5: Uso del Sistema sin Experiencia Técnica

Cuadro 24: Escenario de calidad 5 — Operación por un usuario sin experiencia técnica

Aspecto	**Descripción**
Atributo de calidad	Usabilidad
Requisito	El sistema debe ser utilizable por un usuario con experiencia en neurociencias o análisis de señales, pero sin conocimientos profundos del software.
Fuente del estímulo	Usuario final
Estímulo	El usuario utiliza un plugin complejo como Wavelet, PSD o PAC por primera vez.
Artefacto	Interfaz de usuario, ayudas contextuales, parámetros preconfigurados.
Ambiente	Uso sin acompañamiento técnico.
Respuesta	El usuario puede completar el análisis apoyándose en la interfaz, con guía clara y sin errores operativos.
Medida de Respuesta	Análisis realizado en menos de 5 minutos. Bajo número de consultas al manual de usuario.
Trazabilidad	NFR-U-07 (R42), NFR-U-08 (R43), NFR-U-05 (R10), NFR-U-09 (R44)

6.1.6. Escenario de Calidad 6: Rendimiento con Múltiples Señales Cargadas

Cuadro 25: Escenario de calidad 6 — Rendimiento con múltiples señales cargadas

Aspecto	**Descripción**
Atributo de calidad	Rendimiento
Requisito	El sistema debe mantener tiempos de respuesta estables aun cuando varias señales se encuentren cargadas simultáneamente dentro de la misma ProjectSession.
Fuente del estímulo	Usuario final
Estímulo	El usuario carga entre 2 y 4 señales de gran tamaño y ejecuta diferentes análisis en cada una.
Artefacto	DataStore, gestor de vistas, motor de análisis.
Ambiente	Sistema operando con alto volumen de datos en memoria.
Respuesta	El sistema reutiliza datos sin duplicarlos, mantiene tiempos de acceso aceptables y evita bloqueos en la interfaz.
Medida de Respuesta	Cambio entre señales cargadas en menos de 1 segundo. Sin bloqueos ni cierres inesperados. Uso de memoria dentro del 70–75 %.
Trazabilidad	NFR-P-06 (R51), NFR-P-01/P-08 (R11, R53)

6.1.7. Escenario de Calidad 7: Resiliencia ante Archivos Corruptos o Incompatibles

Cuadro 26: Escenario de calidad 7 — Resiliencia ante archivos corruptos o incompatibles

Aspecto	**Descripción**
Atributo de calidad	Confiabilidad
Requisito	El sistema debe detectar archivos corruptos, incompletos o incompatibles sin comprometer la estabilidad del programa.
Fuente del estímulo	Usuario final o sistema externo
Estímulo	El usuario intenta cargar un archivo ABF/EDF/MAT con estructura inválida o con metadatos corruptos.
Artefacto	FileIOService, Kernel, sistema de alertas.
Ambiente	Sistema en operación normal con señales previamente cargadas.
Respuesta	El sistema captura el error, notifica al usuario, cancela la operación de carga y mantiene intacta la sesión actual.
Medida de Respuesta	Mensaje de error mostrado en menos de 2 segundos. 0 % de pérdida de datos o señales cargadas previamente. El sistema continúa operativo sin fallos.
Trazabilidad	FR-IO-04

6.1.8. Escenario de Calidad 8: Consistencia entre Datos Ingresados y Gráficas Generadas

Cuadro 27: Escenario de calidad 8 — Consistencia entre datos ingresados y gráficas generadas

Aspecto	**Descripción**
Atributo de calidad	Confiabilidad y Usabilidad
Requisito	El sistema debe garantizar que las gráficas generadas correspondan fielmente a los datos y parámetros seleccionados por el usuario.
Fuente del estímulo	Usuario final
Estímulo	El usuario modifica parámetros como canal, ventana temporal, trials seleccionados o rango de frecuencias.
Artefacto	Interfaz de usuario, motor de visualización, servicios de análisis.
Ambiente	Sistema ejecutando análisis consecutivos sobre diferentes configuraciones.
Respuesta	La gráfica generada refleja exactamente los datos filtrados o seleccionados, sin desajustes ni inconsistencias con los parámetros ingresados.
Medida de Respuesta	Actualización gráfica en menos de 3 segundos. 0 diferencias entre los datos mostrados y los procesados. Consistencia confirmada entre ejecuciones sucesivas.
Trazabilidad	NFR-R-04 (R73), NFR-U-02 (R03)

6.2. Reglas de compatibilidad y versionado

- **Versionado de la aplicación.** GammaLab 2.0 adopta versionado semántico `MAJOR.MINOR.PATCH`: `MAJOR` rompe `.glab` o el contrato `IPlugin`; `MINOR` agrega capacidades retrocompatibles; `PATCH` corrige sin alterar contratos públicos. `main.py` expone esta versión en una constante única, visible en un diálogo "Acerca de" y embebida como metadato en cada archivo `.glab` generado.
- **Contrato de plugins (`IPlugin`).** Cambios aditivos (nuevos métodos opcionales, nuevos campos con valor por defecto) no rompen compatibilidad. Cambios que alteren la firma de `initialize`, `start`, `stop`, `process` o `get_widget` elevan la versión `MAJOR` del contrato.
- **Metadatos `properties.yml`.** Los campos requeridos (`id, name, category, subcategory, version, icon, logic_class, ui_class`) deben existir siempre – exigido por `REQUIRED_KEYS` en `loader.py` – y añadir campos opcionales es compatible. El campo `version` declara el `MAJOR` de `IPlugin` contra el que se escribió el plugin; `discover()` rechaza el plugin si no coincide con el `MAJOR` del núcleo en ejecución.
- **Formato de proyecto `.glab`.** `ProjectService` persiste el espacio de trabajo – señal activa, trials y análisis – en archivos `.glab`. Cada `.glab` embebe un `schema_version` propio: un incremento `MINOR` se mantiene legible por versiones futuras de `ProjectService`, y un incremento `MAJOR` rechaza la apertura con un mensaje explícito al usuario en vez de migrar implícitamente.
- **Dependencias externas.** `requirements.txt` fija cada librería con versión exacta (`==`), sin rangos abiertos (Tabla 28). Actualizar VTK, SciPy, PyWavelets o NumPy exige re-ejecutar completas las suites `plugins_test` y `performance_test` antes de aceptar el cambio.
- **Formatos de señal externos (EDF/ABF/BDF/MAT).** La compatibilidad se ancla a la versión de la librería lectora (`pyabf`, `pyedflib`), no al formato en sí. Un archivo no

interpretable por esa versión se reporta como incompatible, sin remuestreo ni heurísticas de recuperación implícitas: un EDF con canales a distinta frecuencia de muestreo falla con un mensaje claro, en vez de intentar corregirse automáticamente.

- **Estabilidad de módulos internos.** Todo renombramiento o reubicación de un módulo público de `core/` actualiza en el mismo cambio a todos sus consumidores, tests incluidos, y un flujo de CI corre `pytest` en cada *push/pull request*. Motivo: un renombramiento previo de `fileio.py` a `fileio_service.py` (con `TrialDataset` movido a `core/model/`) rompió silenciosamente los 8 archivos de `plugins_test` al no actualizarse en el mismo cambio.

Cuadro 28: Versiones de dependencias externas ancladas en `requirements.txt`

Librería	Versión	Rol
`numpy`	2.3.4	Cálculo numérico vectorizado
`scipy`	1.16.2	Filtros, PSD (Welch)
`PyWavelets`	1.9.0	Transformada Wavelet Continua
`vtk`	9.5.2	Renderizado científico 2D/3D
`PyQt5` / `PyQt5-Qt5`	5.15.11 / 5.15.2	Interfaz gráfica
`pyabf`	2.3.8	Lectura de archivos `.abf`
`pyEDFlib`	0.1.42	Lectura de archivos `.edf`
`h5py`	3.15.1	Soporte de formatos basados en HDF5
`pytest`	8.4.2	Framework de pruebas

6.3. Métricas y plan de verificación

La Tabla 29 formaliza la "Medida de Respuesta" de cada escenario de calidad (sección 5.1) en una métrica verificable, junto con su método de medición y el criterio de aceptación correspondiente.

Cuadro 29: Métricas, método de verificación y criterio de aceptación por atributo de calidad

Atributo	Métrica	Método	Criterio de aceptación
Extensibilidad (Esc. 1)	Tiempo de aparición de un plugin nuevo; plugins existentes que fallan	Agregar un plugin válido y correr `plugins_test`	<1 min; 0 plugins fallan (R77–R78)
Rendimiento – carga grande (Esc. 2)	Tiempo a primer render; uso de CPU	`performance_test`; medido: 78.4 ms (1.8 M muestras)	≤8 s estrés, ≤2 s típico; CPU <80 % (R48/R72/R56)
Confiabilidad – errores en ejecución (Esc. 3)	% pérdida de datos; tiempo del mensaje	Inyección de fallos dentro de `process()`	0 % pérdida; mensaje <2 s; sistema operativo
Confiabilidad – exportación (Esc. 4)	Tiempo de exportación; error silencioso	`test_export.py`	<3 s; 0 % pérdida; notificación clara (R51/R57/R70)
Usabilidad (Esc. 5)	Tiempo de análisis sin acompañamiento	Prueba de usabilidad con usuarios reales	<5 min; bajas consultas al manual (R42–R44)

Continúa en la siguiente página

Atributo	Métrica	Método	Criterio de aceptación
Rendimiento – múltiples señales (Esc. 6)	Tiempo de cambio; % de memoria	Benchmark de `DataStore`	<1 s cambio; memoria 70–75 % (R11, R51, R53)
Confiabilidad – archivos corruptos (Esc. 7)	Tiempo del error; continuidad de sesión	`test_fileio_abf/edf.py`	Mensaje <2 s; 0 % pérdida previa (FR-IO-04)
Confiabilidad y Usabilidad (Esc. 8)	Tiempo de actualización; diferencias	Revisión de consistencia entrada-salida	<3 s; 0 diferencias (R73, R03)
Mantenibilidad	% de cobertura automática	`pytest –cov` sobre `core/plugins`	≥80 % de cobertura
Rendimiento – no regresión vs. 1.0	Factor vs. línea base	`compare_report.py`	Nada empeora; render sin vectorizar mejora ≥10×
Portabilidad	Éxito de arranque del `.exe`	Instalación en Windows limpio sin Python	Arranque exitoso, sin configuración manual
Robustez – cancelación (UC-02)	Liberación del hilo tras cancelar	Prueba forzada sobre `TaskOrchestrator`	Liberación garantizada (R46, R70, R74)

6.4. Plan de evidencia

1. **Fixture ABF real y referencias MATLAB.** `test/data/17308005.abf` más los 8 CSV de referencia en `plugins_test/` sustentan la corrección numérica exigida por Extensibilidad (Esc. 1) y por las métricas de Mantenibilidad.

2. **Suite de benchmarks versionada.** `performance_test/`, con resultados persistidos por etiqueta en `perf_results.json` (p. ej. `gammalab1_baseline_2026-08-16`), es la evidencia reproducible de Rendimiento y de la comparación de no regresión frente a 1.0.

3. **Auditorías documentales.** Los informes `reporte_tests.tex` e `informe_rendimiento.tex` (`docs/tests documents/`) registran el estado de la suite de pruebas y los hallazgos de rendimiento.

4. **Pruebas de usabilidad con usuarios reales.** `docs/pruebas-usabilidad/` sustenta el criterio de aceptación de Usabilidad (Esc. 5).

5. **Prueba de instalación limpia.** Arranque del ejecutable generado por PyInstaller en un Windows 10/11 sin Python previamente instalado, como evidencia de Portabilidad.

6. **Registro de cancelación forzada.** La prueba de cancelación forzada sobre el `TaskOrchestrator` bajo carga es la evidencia de Robustez (UC-02).

6.5. Conclusión

Los escenarios de calidad, las reglas de compatibilidad y la tabla de métricas convierten los atributos de calidad de la sección 3.2 en criterios operativos y verificables, no en aspiraciones genéricas. El plan de evidencia de la sección 6.4 sostiene estos criterios con artefactos reproducibles: fixtures reales validados contra una referencia de MATLAB, benchmarks versionados por etiqueta, auditorías documentales, y pruebas de usabilidad y de instalación. Junto con las reglas

de versionado de la sección 6.2, esta estructura garantiza que la arquitectura de GammaLab 2.0 evalúe, documente y demuestre sus atributos de calidad de forma reproducible en cada nueva versión del sistema.

7. Trazabilidad de requerimientos y casos de uso

Esta sección relaciona tres niveles de la especificación: los requerimientos del sistema, los casos de uso que los realizan y las decisiones de arquitectura que los sostienen. Su propósito es permitir dos lecturas opuestas. De los requerimientos hacia los casos de uso, para comprobar que ninguna necesidad del laboratorio quedó sin un comportamiento observable que la atienda. De los casos de uso hacia los requerimientos y las decisiones, para saber qué se compromete en el diseño cuando un caso de uso cambia.

7.1. Fuentes y criterio de construcción

La trazabilidad se construyó a partir de dos artefactos de la especificación: el libro de requerimientos y casos de uso del proyecto, que contiene los 81 requerimientos R01 a R81 con su prioridad y su tipo según FURPS+, y las 23 fichas de casos de uso, cada una con el campo *Requisitos asociados*. Los vínculos entre casos de uso y requerimientos se tomaron literalmente de ese campo; ninguno se infirió por afinidad temática. Los vínculos con la arquitectura se tomaron de la Tabla 11, que ya relaciona cada problema del diagnóstico de la Sección 4 con su principio, su decisión y los casos de uso que verifican su solución. Los casos de uso se agrupan en los cinco subsistemas de la Vista de Escenarios (Sección 5.1).

Un requerimiento se considera *cubierto* cuando al menos un caso de uso lo declara, y un caso de uso se considera *justificado* cuando declara al menos un requerimiento. La Sección 7.5 presenta el resultado de aplicar ambos criterios.

7.2. De los casos de uso a los requerimientos

La Tabla 30 presenta, para cada caso de uso, el subsistema al que pertenece y los requerimientos que realiza. Los casos de uso se ordenan por su identificador.

CU	Nombre	Subsistema	Requerimientos
CU-001	Análisis PAC de un canal	Análisis	R02, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R75
CU-002	Análisis PAC de dos canales / PPC	Análisis	R60, R61, R62, R63, R64, R65, R66
CU-003	Cargar señal (con gestión de memoria)	Núcleo y Ciclo de Vida	R02, R07, R08, R11, R12, R47, R52, R54, R71, R80
CU-004	Generar y modificar trials	Preprocesamiento	R01, R02, R24, R27, R31, R67, R71, R79
CU-005	Guardado y exportación de datos	Exportación	R02, R04, R05, R07, R09, R28, R51, R57
CU-006	Seleccionar trials para análisis	Preprocesamiento	R27, R37, R38, R68
CU-007	Remover artefacto con medidas	Preprocesamiento	R02, R40, R67, R69, R71, R79
CU-008	Navegar y cambiar de módulo	Interfaz y Usabilidad	R06, R43, R45, R50, R55, R58

Continúa en la siguiente página

CU	Nombre	Subsistema	Requerimientos
CU-009	Limpiar formulario (Clear)	Interfaz y Usabilidad	R25
CU-010	Consultar ayuda contextual	Interfaz y Usabilidad	R42
CU-011	Iniciar la aplicación	Núcleo y Ciclo de Vida	R10, R39, R44, R69, R70, R77, R81
CU-012	Interactuar visualmente con la señal	Interfaz y Usabilidad	R26, R30, R32, R49, R59
CU-013	Calcular Wavelet	Análisis	R02, R33, R34, R46, R48, R70
CU-014	Cancelar procesamiento pesado	Núcleo y Ciclo de Vida	R46, R55, R70, R74
CU-015	Cálculo de promedio de trials	Análisis	R03, R36, R41, R56, R71
CU-016	Aplicar filtros digitales	Preprocesamiento	R58, R59, R71, R76, R79
CU-017	Exportación gráfica vectorial (EMF)	Exportación	R28, R35, R57
CU-018	Ajustar frecuencia de muestreo	Preprocesamiento	R29, R67, R72
CU-019	Optimizar renderizado (submuestreo)	Interfaz y Usabilidad	R30, R49, R59, R73
CU-020	Orquestador del núcleo (ejecutar análisis pesado)	Núcleo y Ciclo de Vida	R04, R46, R54, R55, R69, R70, R71, R77
CU-021	Cerrar un proyecto	Núcleo y Ciclo de Vida	R02, R07, R11, R12, R52
CU-022	Cerrar la aplicación	Núcleo y Ciclo de Vida	R45, R46, R53, R58
CU-023	Abrir un proyecto	Núcleo y Ciclo de Vida	R02, R07, R08, R39, R47, R54, R71, R78

Cuadro 30: Requerimientos realizados por cada caso de uso

7.3. De los requerimientos a los casos de uso

La Tabla 31 presenta la lectura inversa: para cada requerimiento, su tipo, su prioridad y los casos de uso que lo realizan. El enunciado completo de cada requerimiento se encuentra en el libro de requerimientos y en el SRS, y no se repite aquí para que la tabla sirva como índice.

R	Tipo (FURPS+)	Prioridad	Casos de uso
R01	Funcional	Alta	CU-004
R02	Usabilidad	Media	CU-001, CU-003, CU-004, CU-005, CU-007, CU-013, CU-021, CU-023
R03	Usabilidad	Media	CU-015
R04	Usabilidad	Alta	CU-005, CU-020
R05	Funcional	Alta	CU-005
R06	Usabilidad	Media	CU-008
R07	Funcional	Alta	CU-003, CU-005, CU-021, CU-023
R08	Funcional	Alta	CU-003, CU-023
R09	Funcional	Baja	CU-005
R10	Usabilidad	Baja	CU-011

Continúa en la siguiente página

R	Tipo (FURPS+)	Prioridad	Casos de uso
R11	Rendimiento	Alta	CU-003, CU-021
R12	Usabilidad	Media	CU-003, CU-021
R13	Funcional	Alta	CU-001
R14	Funcional	Alta	CU-001
R15	Funcional	Alta	CU-001
R16	Funcional	Alta	CU-001
R17	Funcional	Alta	CU-001
R18	Funcional	Alta	CU-001
R19	Funcional	Media	CU-001
R20	Funcional	Media	CU-001
R21	Funcional	Media	CU-001
R22	Funcional	Media	CU-001
R23	Funcional	Media	CU-001
R24	Funcional	Alta	CU-004
R25	Funcional	Baja	CU-009
R26	Funcional	Media	CU-012
R27	Funcional	Alta	CU-004, CU-006
R28	Funcional	Baja	CU-005, CU-017
R29	Funcional	Alta	CU-018
R30	Funcional	Alta	CU-012, CU-019
R31	Funcional	Alta	CU-004
R32	Funcional	Media	CU-012
R33	Funcional	Alta	CU-013
R34	Funcional	Alta	CU-013
R35	Funcional	Baja	CU-017
R36	Funcional	Alta	CU-015
R37	Funcional	Alta	CU-006
R38	Funcional	Media	CU-006
R39	Soporte	Baja	CU-011, CU-023
R40	Funcional	Baja	CU-007
R41	Funcional	Alta	CU-015
R42	Usabilidad	Alta	CU-010
R43	Usabilidad	Alta	CU-008
R44	Usabilidad	Media	CU-011
R45	Rendimiento	Alta	CU-008, CU-022
R46	Confiabilidad	Alta	CU-013, CU-014, CU-020, CU-022
R47	Rendimiento	Alta	CU-003, CU-023
R48	Rendimiento	Baja	CU-013
R49	Rendimiento	Alta	CU-012, CU-019
R50	Rendimiento	Alta	CU-008
R51	Rendimiento	Media	CU-005
R52	Rendimiento	Media	CU-003, CU-021
R53	Rendimiento	Media	CU-022
R54	Confiabilidad	Alta	CU-003, CU-020, CU-023
R55	Rendimiento	Media	CU-008, CU-014, CU-020
R56	Rendimiento	Media	CU-015
R57	Rendimiento	Media	CU-005, CU-017
R58	Confiabilidad	Alta	CU-008, CU-016, CU-022
R59	Rendimiento	Media	CU-012, CU-016, CU-019
R60	Funcional	Alta	CU-002
R61	Funcional	Alta	CU-002
R62	Funcional	Alta	CU-002
R63	Funcional	Media	CU-002

Continúa en la siguiente página

R	Tipo (FURPS+)	Prioridad	Casos de uso
R64	Funcional	Media	CU-002
R65	Funcional	Media	CU-002
R66	Funcional	Media	CU-002
R67	Soporte	Alta	CU-004, CU-007, CU-018
R68	Funcional	Alta	CU-006
R69	Soporte	Alta	CU-007, CU-011, CU-020
R70	Soporte	Alta	CU-011, CU-013, CU-014, CU-020
R71	Rendimiento	Alta	CU-003, CU-004, CU-007, CU-015, CU-016, CU-020, CU-023
R72	Confiabilidad	Alta	CU-018
R73	Rendimiento	Alta	CU-019
R74	Rendimiento	Alta	CU-014
R75	Funcional	Media	CU-001
R76	Funcional	Alta	CU-016
R77	Soporte	Alta	CU-011, CU-020
R78	Funcional	Alta	CU-023
R79	Confiabilidad	Alta	CU-004, CU-007, CU-016
R80	Funcional	Alta	CU-003
R81	Rendimiento	Media	CU-011

Cuadro 31: Casos de uso que realizan cada requerimiento

7.4. De los casos de uso a las decisiones arquitectónicas

La Tabla 32 invierte la Tabla 11: indica, para cada caso de uso que sirve de verificación a una decisión de la Sección 4, el problema del diagnóstico al que responde, los principios y las decisiones que lo atienden y, cuando existe, el escenario de calidad asociado. Un caso de uso que cambie obliga a revisar las decisiones de su fila.

CU	Problema	Principio	Decisión	Escenario de calidad
CU-003	D2, D5	P2, P5, P7	ADR-03, ADR-06, ADR-08	Escenario 6
CU-004	D2	P2	ADR-03	—
CU-007	D7	P5	ADR-06	Escenario 8
CU-008	D8	P7	ADR-08	—
CU-011	D1, D4, D14	P1, P4, P10	ADR-01, ADR-02, ADR-05, ADR-11	Escenarios 1 y 3
CU-014	D11	P8	ADR-09	—
CU-016	D7	P5	ADR-06	Escenario 8
CU-019	D6	P6	ADR-07	Escenarios 2 y 6
CU-020	D9, D10, D12	P8, P9	ADR-09, ADR-10	Escenario 2
CU-021	D5	P5, P7	ADR-06, ADR-08	Escenario 6
CU-022	D1, D8	P1, P7	ADR-01, ADR-02, ADR-08	—

Cuadro 32: Decisiones arquitectónicas que sostienen cada caso de uso

Los casos de uso CU-001, CU-002, CU-005, CU-006, CU-009, CU-010, CU-012, CU-013, CU-015, CU-017, CU-018 y CU-023 no figuran en la tabla porque ninguno de los problemas del diagnóstico de la versión 1.0 se verifica a través de ellos. Se realizan sobre la estructura general del microkernel y de los servicios del núcleo, sin una decisión arquitectónica propia.

7.5. Análisis de cobertura

La Tabla 33 resume el resultado de aplicar los criterios de la Sección 7 a las dos matrices anteriores.

Verificación	Resultado	Proporción
Requerimientos declarados por al menos un caso de uso	81 de 81	100 %
Casos de uso que declaran al menos un requerimiento	23 de 23	100 %
Requerimientos declarados por más de un caso de uso	26 de 81	32 %
Referencias a requerimientos inexistentes en el libro	0	—

Cuadro 33: Cobertura de la trazabilidad entre requerimientos y casos de uso

Los 81 requerimientos se distribuyen así según FURPS+: 45 funcional, 17 rendimiento, 9 usabilidad, 5 soporte, 5 confiabilidad. Los casos de uso que más requerimientos concentran son CU-001 (13 requerimientos), CU-003 (10 requerimientos), CU-023 (8 requerimientos), CU-020 (8 requerimientos), CU-005 (8 requerimientos), CU-004 (8 requerimientos), lo que los hace los más sensibles a un cambio de alcance. Los requerimientos más compartidos son R02 (8 casos de uso), R71 (7 casos de uso), R70 (4 casos de uso): son requerimientos transversales, que se cumplen en varios casos de uso a la vez y deben verificarse en cada uno.

Dos observaciones completan el análisis. Primero, la cobertura completa significa que cada requerimiento tiene un caso de uso que lo realiza, pero no que esté implementado ni verificado; la verificación de los atributos de calidad se sigue en la Sección 6.3. Segundo, CU-020 (ejecución a través del orquestador) es incluido por diez casos de uso en el diagrama de la Figura 2, por lo que los requerimientos que declara (R04, R46, R54, R55, R69, R70, R71 y R77) condicionan indirectamente a todos ellos.

8. Anexos

8.1. Anexo A — Relación con el SRS

En particular:

-

8.2. Anexo B — Relación con el SDD

Este anexo resume dicha relación:

-

8.3. Anexo C — Trazabilidad global de artefactos

8.4. Anexo D — Diagramas y tablas de referencia

9. Referencias

Referencias

[1] Python Software Foundation. *Python 3 — The Python Standard Library*. Disponible en: https://docs.python.org/3/.

[2] Riverbank Computing. *PyQt5 Reference Guide*. Disponible en: https://www.riverbankcomputing.com/static/Docs/PyQt5/.

[3] The Qt Company. *Qt 5 — Style Sheets Reference*. Disponible en: https://doc.qt.io/qt-5/stylesheet-syntax.html.

[4] Kitware, Inc. *VTK — Visualization Toolkit Documentation*. Disponible en: https://vtk.org/ y https://docs.vtk.org/.

[5] Kitware, Inc. *vtkChartXY Class Reference*. Disponible en: https://docs.vtk.org/en/latest/api/python/vtkmodules.vtkChartsCore.html#vtkmodules.vtkChartsCore.vtkChartXY.

[6] NumPy Developers. *NumPy Documentation*. Disponible en: https://numpy.org/doc/.

[7] Scott W. Harden. *pyABF — Read and Analyze ABF Files with Python*. Disponible en: https://pyabf.readthedocs.io/.

[8] Holger Nahrstaedt et al. *pyEDFlib — Python library to read/write EDF+/BDF+ files*. Disponible en: https://pyedflib.readthedocs.io/.

[9] Oren Ben-Kiki, Clark Evans, Ingy döt Net. *YAML Ain't Markup Language (YAML) Specification*. Disponible en: https://yaml.org/spec/.

[10] P. B. Kruchten. *The 4+1 View Model of Architecture*. IEEE Software, vol. 12, no. 6, pp. 42–50, 1995. DOI: https://doi.org/10.1109/52.469759.

[11] Len Bass, Paul Clements, Rick Kazman. *Software Architecture in Practice* (3rd ed.). Addison–Wesley, 2012. ISBN 978-0-321-81573-6.

[12] Paul Clements, Felix Bachmann, Len Bass, David Garlan, James Ivers, Reed Little, Robert Nord, Judith Stafford. *Documenting Software Architectures: Views and Beyond* (2nd ed.). Addison–Wesley, 2010. ISBN 978-0-321-55268-6.

[13] Rick Kazman, Mark Klein, Paul Clements. *ATAM: Method for Architecture Evaluation*. CMU/SEI-2000-TR-004, Software Engineering Institute, 2000.

[14] Bob Kemp, Alpo Värri, Agostinho Rosa, Kim D. Nielsen, John Gade. *A simple format for exchange of digitized polygraphic recordings*. Electroencephalography and Clinical Neurophysiology, 82(5), 1992, pp. 391–393. DOI: https://doi.org/10.1016/0013-4694(92)90009-7.

[15] P. L. Nunez, R. Srinivasan. *Electric Fields of the Brain: The Neurophysics of EEG* (2nd ed.). Oxford University Press, 2006.

[16] Equipo Gamma Lab. *Software Architecture Document (SAD), v1.0*. Este documento.

[17] Object Management Group. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. OMG Doc. formal/17-12-05, dic. 2017. Disponible en: https://www.omg.org/spec/UML/2.5.1/.

Nota: Las URLs señaladas se incluyen para consulta rápida. En publicación impresa, usar DOI/ISBN cuando aplique.